

King Saud University
College of Applied Studies and Community Services
Department of Computer Science and Engineering
Bachelor's Degree in Applied Computing – Cybersecurity Track

CamDecept: An Adaptive Honeypot Framework for IoT Surveillance Devices

Project Submitted in Fulfillment of the CYB 496 Course requirements.

By:

Group#3

Dunia Alshahrani

Ohoud Alshehri

Maysoon Alkhaldi

Sara Salman

Layan Alshmmasi

Buthaina Alhajlah

Jana Aldawas

Supervised by:

Dr. Nada Alhirabi

Academic Year 2025-2026

Declaration

We Dunia Alshahrani, Layan Alshmmasi, Maysoon Alkhaldi, Sara Salman, Ohoud Alshehri, Buthaina Alhajlah, and Jana Aldawas, being members of final year project group number three, declare that this report contains only work completed by members of our group except for information obtained legitimately from literature, company or university sources. All information from these other sources has been duly referenced and acknowledged by the University Policy on Plagiarism.

Furthermore, we declare that in completing the project, the individual group members had the following responsibilities and contributed in the following proportions to the outcomes of the project:

Responsibilities	Contribution %	Signature
Chapter 6&5	100%	Dunia
Chapter 5&6	100%	Ohoud
Chapter3	100%	Jana
Chapter 7&8	100%	Buthaina

Chapter 4&2	100%	Layan
Chapter 1&3	100%	Maysoon
Chapter 4	100%	Sara

Acknowledgment

We would like to sincerely thank Almighty Allah for His abundant blessings, guidance, and strength that enabled us to successfully complete this project. We would like to express our deep appreciation to our supervisor, Ms. Nada Alhirabi, for her continuous assistance, valuable feedback, and expert guidance throughout this project. Her empathy, support, and insightful advice greatly enhanced the effectiveness and excellence of our efforts. We would like to express our gratitude to the faculty members of our department for their support and assistance throughout the project's implementation and assessment. We sincerely appreciate our family and friends for their continuous encouragement and support during this project and throughout our years of study

Abstract

The increasing exploitation of IP cameras in IoT environments highlights the need for specialized security mechanisms capable of analyzing attacker behavior rather than merely recording it. This project presents CamDecept, an adaptive honeypot framework designed to simulate IP camera services and capture real attacker interactions in a controlled environment. The system emulates realistic HTTP and RTSP endpoints, logs all incoming requests, and applies a multi-rule detection engine combined with a risk-scoring model to classify threats such as brute-force attempts, unauthorized access, scanning behavior, and automated tool activity. Experimental evaluation—based on real attack simulations—demonstrates the framework’s ability to detect malicious behavior early, distinguish between low- and high-risk activities, and provide meaningful behavioral insights. By integrating deception, behavior analysis, and risk scoring into a unified architecture, CamDecept enhances understanding of attacker tactics and offers a practical foundation for improving IP camera security in IoT ecosystems.

Keywords: IoT Security, IP Cameras, Honeypots, Cyber Deception, Risk Scoring, Behavioral Analysis

Abstract (in Arabic)

يشهد إنترنت الأشياء تزايداً ملحوظاً في الهجمات التي تستهدف كاميرات المراقبة المتصلة بالشبكة، مما يبرز الحاجة إلى حلول أمنية متخصصة قادرة على تحليل سلوك المهاجمين بدلاً من الاكتفاء بتسجيل محاولاتهم. يقدم هذا المشروع إطاراً خداعياً متكاملاً يحمل اسم «كام ديسيب» ، يهدف إلى محاكاة بيئة كاميرات المراقبة بشكل واقعي لجذب المهاجمين وتحليل تفاعلاتهم داخل بيئة آمنة ومعزولة. يعتمد النظام على محاكاة واجهات الكاميرا الشائعة، وتسجيل جميع الطلبات الواردة، ثم تحليلها باستخدام محرك كشف يعتمد على مجموعة من القواعد ونموذج لتقييم مستوى الخطورة، مما يتيح تصنيف التهديدات مثل محاولات التخمين، والوصول غير المصرح به، والسحب والاستكشاف، واستخدام الأدوات الآلية. أظهرت التجارب الميدانية على محاكاة هجمات فعلية قدرة النظام على الاكتشاف المبكر، والتمييز بين الأنشطة منخفضة وعالية الخطورة، وتقديم رؤية دقيقة حول سلوك المهاجمين. يجمع «كام ديسيب» بين الخداع والتحليل السلوكي وتقييم المخاطر في إطار واحد، مما يعزز فهم أساليب الهجوم ويدعم تطوير حلول أكثر فعالية لحماية كاميرات المراقبة. في بيئات إنترنت الأشياء.

الكلمات المفتاحية: أمن إنترنت الأشياء، كاميرات المراقبة، الفخاخ الافتراضية، الخداع السيبراني، تقييم المخاطر، التحليل السلوكي

List of Abbreviations

MQTT	Message Queuing Telemetry Transport
API	Application Programming Interface
IoTPOT	Internet of Things Honeypot
USENIX WOOT'15	USENIX Workshop on Offensive Technologies, 2015
CPU architectures	Central Processing Unit Architectures (e.g., ARM, MIPS)
IoT	Internet of Thing
SSH	Secure Shell
Nmap	Network Mapper
Telnet	Telecommunication Network
ICS	Industrial Control Systems
TTPs	Tactics, Techniques, and Procedures
ML	Machine Learning
CoAP	Constrained Application Protocol
BLE	Bluetooth Low Energy
SCADA	Supervisory Control And Data Acquisition
MDP	Markov Decision Process
APT	Advanced Persistent Threat
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
HTTP	Hypertext Transfer Protocol

Table of Contents

Declaration.....	ii
Acknowledgment.....	iv
Abstract.....	v
List of Figures	xi
List of Tables	xii
1. Introduction	2
1.1 Motivation	2
1.2 Problem Definition.....	3
1.3 State of the Art and Research Gap.....	3
1.4 Proposed Idea.....	4
1.5 Aims and Objectives	4
1.6 Project Scope.....	5
1.7 Project’s Contributions.....	6
1.8 Report Structure	6
2.2 IP Camera Vulnerabilities	9
2.3 Honeypots & Cyber Deception	10
2.4 Security Behavior Analysis.....	11
3. Related work & Literature Review	14
3.1 IoT Honeypot-Based Attack Detection.....	14
3.2 Smart and Adaptive Honeypots	14
3.3 Attack Detection in IoT Environments	15
3.4 Comparative Analysis of Existing Systems.....	15
4.1 System Overview.....	19
4.4 Abuse Cases Scenarios.....	21

5 Implementation	32
5.1 System Architecture Design	32
5.2 Components Description	35
5.2.1 cam_2.py - Honeypot Flask Server	35
5.3 Detection Mechanism	37
5.4 Risk Scoring Model	39
5.5 Implementation Details	41
5.6 Logging and Monitoring	43
5.6.1 Request Log (logs.jsonl)	44
5.7 Alert Generation	45
Chapter 6	52
Evaluation and Results	52
6 Results and Evaluation	53
6.1 Experimental Setup	53
6.2 Test Scenarios	53
6.2.1 Browser Reconnaissance Scenario (TS1)	55
6.2.2 Unauthorized Admin Access Scenario (TS2)	55
6.2.3 Automated PowerShell Attack Scenario (TS3)	55
6.2.4 Brute-Force Authentication Scenario (TS4)	56
6.2.5 Multi-Path Scanning Scenario (TS5)	56
6.2.6 Unauthorized Stream Access Scenario (TS6)	57
Scenario Evaluation Summary	57
6.3 Evaluation Metrics	57
6.4 Experimental Results	58
6.5 Attack Detection Analysis	59
6.6 Risk Scoring Evaluation	59

6.7 Discussion of Results.....	60
6.8 Evaluation Summary	61
Chapter 7	62
Discussion.....	62
7.1	63
8.1 Conclusion.....	66
8.2 Future Work	66
References	67

List of Figures

FIGURE 1 ARCHITECTURE OF AN IP CAMERA HONEYPOT SYSTEM [7]	11
FIGURE 2 IP CAMERA HONEYPOT ATTACK FLOW AND DETECTION PROCESS	23
FIGURE 3 ABUSE CASE DIAGRAM OF THE CAMDECEPT FRAMEWORK	24
FIGURE 4 ATTACK AND DEFENSE DIAGRAM OF CAMDECEPT	25
FIGURE 5 SHOWS A THREAT MODEL FOR AN IP CAMERA HONEYPOT SYSTEM, INCLUDING ITS COMPONENTS, POSSIBLE ATTACKS, AND SECURITY PROTECTIONS	26
FIGURE 6 NETWORK AND COMMUNICATION ARCHITECTURE	28
FIGURE 7 LIVE HTTP RESPONSES CAPTURED	36
FIGURE 8 AI_ANALYZER.PY INITIALIZATION	38
FIGURE 9 DETECTION RULES R1, R2, AND R3 WITH THEIR SCORE CONTRIBUTIONS	38
FIGURE 10 DETECTION RULES R4, R5, AND R6 WITH THEIR SCORE CONTRIBUTIONS	39
FIGURE 11 MDP STATE TRANSITION PROBABILITY FORMULA	39
FIGURE 12 LIVE ATTACK SIMULATION	42
FIGURE 13 CAMDECEPT DEVELOPMENT AND TESTING ENVIRONMENT SPECIFICATIONS	43
FIGURE 14 LOGS.JSONL ENTRIES SHOWING BROWSER RECONNAISSANCE TRANSITIONING TO POWERSHELL ATTACK TRAFFIC	44
FIGURE 15 DUAL-LOG WRITE LOGIC: LOG_FILE FOR ALL REQUESTS, ALERT_FILE FOR MEDIUM/HIGH ONLY	45
FIGURE 16 FLASK TERMINAL	46
FIGURE 17 DASHBOARD ALL LOGS TABLE SCREENSHOT	47
FIGURE 18 DASHBOARD OVERVIEW + ATTACK TYPES CHART	48
FIGURE 19 RISK LEVELS CHART	49

List of Tables

TABLE 1 ANALYSIS OF ATTACK TYPES AND ATTACKER ACTIVITIES.....	15
TABLE 2 COMPARISON OF EXISTING IOT AND IP CAMERA HONEYPOT SYSTEMS.....	16
TABLE 3 CAMDECEPT THREE-LAYER ARCHITECTURE	33
TABLE 4 RISK LEVEL	40
TABLE 5 SCORE PROGRESSION ACROSS SIMULATED ATTACK SCENARIOS.....	41
TABLE 6 EVALUATION OF THE IMPLEMENTATION.....	58

Chapter 1

Introduction

1. Introduction

The rapid expansion of Internet of Things (IoT) devices has transformed modern homes, workplaces, and public environments. Among these devices, IP cameras have become one of the most widely deployed technologies due to their convenience, low cost, and ability to provide continuous monitoring. However, this widespread adoption has also introduced serious security and privacy challenges. Many IP cameras suffer from weak authentication, outdated firmware, exposed services, and limited built-in protection mechanisms. As a result, attackers frequently target these devices to gain unauthorized access, intercept video streams, or even recruit them into large-scale botnets.

Traditional security solutions often fail to detect or understand these attacks in depth. Honeypots have been used to observe attacker behavior, but most existing systems focus mainly on collecting data rather than analyzing it intelligently. They rarely provide real-time insights, do not infer attacker intent, and often ignore privacy considerations. This creates a gap between what current honeypots offer and what is needed to protect IP cameras in real environments.

This project introduces **CamDecept: An Adaptive Honeypot Framework for IoT Surveillance Devices**, a privacy-aware intelligent honeypot designed specifically for IP camera security. The system not only captures attacker interactions but also analyzes behavior, classifies attack types, infers intent, and generates simulated response recommendations. By integrating privacy-aware logic, the system ensures that no real user data is exposed during the process.

1.1 Motivation

Cameras have become a common target for attackers because they provide direct access to sensitive visual information. When compromised, these devices can be used for unauthorized monitoring, eavesdropping, or manipulation of the camera's functions, creating significant security and privacy risks in homes, workplaces, and critical environments.

The motivation behind this project arises from the need to better understand how attackers behave when interacting with IP cameras. Instead of simply recording attacks, there is a need for a system capable of analyzing attacker interactions and generating meaningful insights. A rule-based honeypot can help researchers and security teams identify attack patterns, understand attacker goals, and design more effective defenses.

Another motivation is the lack of privacy-aware honeypot solutions. Many existing systems rely on real camera feeds or sensitive data, which introduces ethical and privacy concerns. This project aims to create a safe environment using synthetic streams and privacy-focused logic to ensure that no real individuals or real video data are exposed.

1.2 Problem Definition

Despite the availability of honeypots for IoT devices, there is still no unified system that combines intelligent analysis, attacker intent inference, and privacy-aware response generation specifically for IP cameras. Existing solutions suffer from several limitations:

- They mainly collect data without providing deeper reasoning.
- They lack real-time or near real-time analysis.
- They do not generate actionable mitigation strategies.
- They treat privacy as a separate issue rather than integrating it into the detection process.
- Many honeypots are not specialized for IP cameras and therefore miss camera-specific attack patterns.

The problem addressed in this project is the absence of an intelligent, privacy-aware honeypot that can detect, classify, and analyze IP camera attacks while also generating simulated response recommendations.

1.3 State of the Art and Research Gap

Several studies have explored IoT honeypots, camera deception systems, and AI-based behavior analysis. For example, SIPHON and HoneyCam focus on capturing real-world

attacks but lack intelligent reasoning. IoT CandyJar and AIIPot introduce machine learning, but they are not designed for IP cameras and often require heavy training. Other systems like CamDec simulate camera behavior but do not include AI-driven analysis or privacy-aware logic.

From the reviewed literature, it is clear that:

- Honeypots exist, but they are mostly passive.
- AI-based systems exist, but they are not camera-specific.
- Privacy-focused studies exist, but they do not integrate with honeypots.

No existing work combines intelligent analysis, a camera-specific honeypot, and privacy-aware response generation in a single framework. This gap is what CamDecept aims to fill.

1.4 Proposed Idea

The proposed system, CamDecept, is an intelligent honeypot designed to simulate an IP camera environment and analyze attacker behavior. The system includes:

- A realistic IP camera simulation (HTTP, RTSP-like endpoints).
- A data collection layer that logs login attempts, requests, and session behavior.
- A behavior analysis engine that extracts features and classifies attacks.
- An intent inference module that determines whether the attacker aims to spy, compromise the device, or recruit it into a botnet.
- A simulated response engine that recommends mitigation strategies.
- A privacy-aware layer that ensures no real data is used, and that all outputs consider privacy risks.

This idea transforms the honeypot from a passive trap into an intelligent decision-support tool.

1.5 Aims and Objectives

The main aim is to design and implement a privacy-aware, rule-based honeypot system that detects IP camera attacks, analyzes attacker behavior, and generates simulated mitigation responses.

Specific Objectives:

1. Develop a realistic IP camera honeypot environment.
2. Capture and log attacker interactions.
3. Analyze attacker behavior using a rule-based behavioral analysis engine and a risk-scoring detection model.
4. Classify attack types and infer attacker intent.
5. Generate simulated response recommendations.
6. Integrate privacy-aware considerations into the detection and response process.

1.6 Project Scope

This project focuses on simulating an IP camera environment rather than using real hardware. The honeypot will support basic camera-related protocols such as HTTP and RTSP-like endpoints. The system will collect attacker interactions, analyze them, and generate simulated responses.

The Scope includes:

- Simulating login pages, video streams, and camera interfaces.
- Capturing brute-force attempts, unauthorized access, RTSP abuse, and command injection.
- Using synthetic video streams to avoid privacy risks.
- Implementing a dashboard to display attack details and recommendations.

The project does not aim to block attacks in real time or replace commercial security systems. Instead, it provides insights and recommendations for research and educational purposes.

1.7 Project's Contributions

This project contributes to the field in several ways:

- A specialized honeypot designed specifically for IP cameras.
- Integration of intelligent behavior analysis rather than simple logging.
- A system that infers attacker intent, not just attack type.
- A simulated response engine that maps attacks to mitigation strategies.
- A privacy-aware design that avoids the use of real video data.
- A unified framework that combines detection, analysis, and privacy in one system.

These contributions help bridge the gap between traditional honeypots and modern intelligent security systems.

1.8 Report Structure

This report is organized into several chapters, each addressing a specific part of the project. The structure is as follows:

- Chapter 1 – Introduction:

Provides an overview of the project, including the motivation, problem definition, research gap, proposed idea, aims and objectives, project scope, and the main contributions.

- Chapter 2 – Literature Review:

Discusses existing studies related to IP camera security, honeypots, attacker behavior analysis, and privacy-aware systems. It highlights the strengths and limitations of previous work and positions the project within the current research landscape.

- Chapter 3 – Methodology:

Explains the design and architecture of the proposed system, including the honeypot environment, data collection process, behavior analysis techniques, simulated response engine, and privacy-aware components.

- Chapter 4 – Implementation:

Describes how the system was built, the technologies used, the development process, and the integration of different components. It also includes screenshots, code structure, and system workflow.

- Chapter 5 – Results and Evaluation:

Presents the experimental results, detection performance, classification accuracy, system behavior under different attack scenarios, and evaluation of privacy considerations.

- Chapter 6 – Conclusion and Future Work:

Summarizes the project outcomes, discusses the main findings, and suggests possible improvements or extensions for future research.

Chapter 2

Background

This chapter presents the background of the CamDecept framework, including IoT security, IP camera vulnerabilities, and honeypot technologies related to cyber-attacks.

2.1 IOT Security Overview

Internet of things has been fast-growing in both smart home applications, enterprise, and industry sectors. Devices in these networks are normally online and therefore are very vulnerable to any form of attack on the network. Nevertheless, most of IoT devices are usually vulnerable owing to factors like lack of computing capacity, insufficient updates, and poor authentication protocols.

Several research findings indicate that IoT devices are among the most attacked devices since they are normally exposed, with poor configurations. The use of botnets as an attack strategy is also evident. In particular, the use of IP cameras within the IoT devices has shown that a lot of devices are vulnerable to cyber-attacks, as found by IoT POT analysis.

IP cameras within IoT devices are extremely vulnerable due to the reason that they are always online and have remote connection features

2.2 IP Camera Vulnerabilities

There exist various shortcomings that can be attributed to IP camera security, which makes these devices vulnerable to attacks. The security problems are associated with authentication problems, exposure of administrative interfaces, and unsafe protocols used.

According to the study called "SweetCam: an IP Camera Honeypot," it has been identified that IP cameras are often attacked due to the poor authentication processes that the devices possess and their use of insecure protocols like HTTP and RTSP. Another problem is the availability of the cameras since they are online all the time and thus vulnerable to automated scanning.

IP cameras are vulnerable to several types of cyberattacks due to weak authentication mechanisms, exposed administrative interfaces, and the use of insecure protocols such as HTTP and RTSP. According to the study "SweetCam: an IP Camera Honeypot," attackers often target IP cameras through brute-force attacks, unauthorized remote access attempts, RTSP stream abuse, malware injection, and automated scanning activities. In addition, many IP cameras remain continuously connected to the internet and rely on default or weak credentials, which increases the risk of unauthorized access and privacy breaches. These security challenges highlight the need for dedicated protection mechanisms and deception-based frameworks such as CamDecept to detect and analyze malicious activities targeting IP camera systems.

2.3 Honeypots & Cyber Deception

Honeypots are cybersecurity measures which create a realistic environment to lure attackers for study purposes. They do not prevent any attacks from happening but rather serve as deception targets to learn about the strategies used by hackers.

According to the ThingPot paper, honeypots replicate real-life devices and collect evidence such as traffic patterns, commands, and attack techniques . In other words, using honeypots is one way of learning more about attackers' techniques and discovering new attack methods.

Likewise, the paper on IoT POT proves that honeypots are capable of collecting malware samples and detecting different forms of attacks, including DDoS attacks .

Using honeypots in IP cameras can help in the following ways:

- Detecting attacks early
- Learning about attack techniques
- Interacting with attackers without endangering the system

As shown in Figure 1, the SweetCam architecture offers IP camera honeypots that mimic three primary services – HTTP, RTSP, and SSH. The HTTP service mimics the website, which includes the login and camera feed pages, whereas RTSP services simulate video streaming. Furthermore, the SSH service works with Cowrie honeypot to collect attacker's commands. This architecture supports the idea of our project by showing how fake IP camera services can be used to attract attackers and collect their behavior safely.

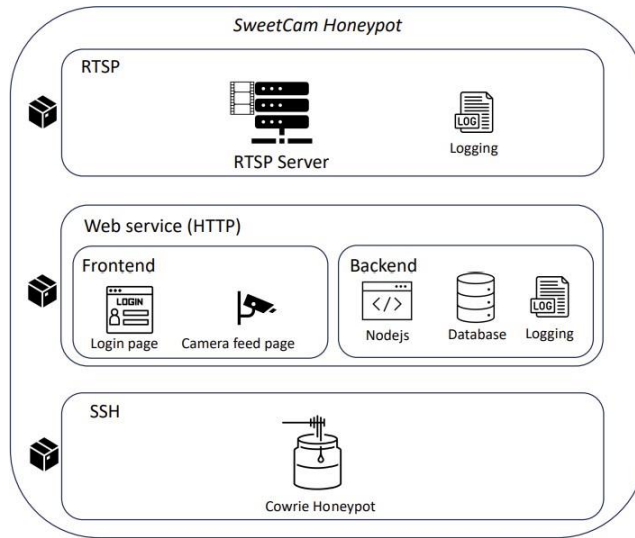


Figure 1 Architecture of an IP camera honeypot system [7].

2.4 Security Behavior Analysis

Security behaviour analysis refers to the analysis of the way in which attackers engage with systems rather than detection of attacks alone. This involves the study of activities like login attempts, execution of commands, and access patterns.

The following attacker behaviours can be obtained from honeypot studies using the logs that comprise of:

- Source IP addresses
- Login attempts
- Commands executed
- Services accessed

For instance, it is records attacker interactions and identifies different types of attack behaviours and patterns.

By analyzing this information, security measures can:

- Identify suspicious patterns
- Categorize attacks types
- Improve detection accuracy

This makes behavioral analysis an essential component in the modern deception-based that in security frameworks.

2.5 Risk Scoring Concepts

Risk scoring is a concept that helps assess the severity of the attack based on some predefined criteria. In this approach, not all attacks receive equal consideration; rather, their significance varies according to specific behaviors of the attacker.

When dealing with honeypots, risk scoring might include the following considerations:

- Number of login failures
- Access attempt frequency
- The type of service attacked (for example, RTSP versus HTTP) Execution of any suspicious commands

Based on these and other criteria, the risk level of the threat can be established (for example, low, moderate, or high).

Benefits of risk scoring include:

- Efficient prioritization of critical threats
- False positive reduction
- Automatic decision making

In the proposed IP camera deception framework, the risk scoring plays a key role in the identifying high-risk attackers also improving overall of security monitoring.

Chapter 3

Literature Review

3. Related work & Literature Review

3.1 IoT Honeypot-Based Attack Detection

Honeypots have turned out to be critical tools that can be used for the detection and analysis of attacks against Internet of Things (IoT). Honeypots create a simulation of vulnerable systems to lure attackers and to collect useful information about their actions and attack tactics. High-interaction honeypots, such as SIPHON, have proven to be efficient in capturing actual cyberattacks, including brute-force attacks, malware distribution, and access to IoT devices [1]. Moreover, IoTcandyJar and ThingPot have been created to mimic IoT service and protocols and to track down interactions between attackers and IoT devices [2], [3]. These systems shed light on some types of attacks that exploit weak credentials, use scanning to detect potential targets, and recruit botnets. For IP cameras, there are special honeypots called SweetCam and HoneyCam that are capable of capturing actual attacks on camera-specific services and protocols, including RTSP and SSH [7], [13]. They make it possible to analyze some of the attacks against cameras, such as video manipulation and accessing surveillance feeds. Nevertheless, most available honeypot systems were designed specifically to monitor IoT attacks rather than IP camera-based attacks [4], [15].

3.2 Smart and Adaptive Honeypots

To improve the effectiveness of honeypots, there have been several research efforts dedicated to developing intelligent and adaptive honeypots to facilitate the interaction between the systems and hackers in an intelligent manner.

An example of this kind is HoneyIoT, which utilizes reinforcement learning ideas to make the system react adaptively at runtime [8], [12]. In another approach, AIIPot incorporates intelligent interactions using machine learning methods to model the session of attackers and extend the interaction period [10].

In addition to adaptive IoT honeypot ecosystem technologies that observe and analyze the behavior of the attacker to adjust the system response, there are other systems such as AIIPot. Such adaptive measures greatly contribute to the collection of more useful information and behavioral analysis.

However, despite all the improvements made in the creation of honeypots, very few of the systems are adaptive and work specifically on IP camera protocols such as RTSP and ONVIF.

3.3 Attack Detection in IoT Environments

Attack detection in IoT systems usually involves monitoring the network traffic and user behavior, as well as system log files. Detection approaches are generally based on predefined rules, such as high frequency of requests, repeated unsuccessful attempts at login, and other forms of unauthorized access attempts. Research in recent years has revealed that machine learning approaches can help considerably increase the efficiency of detecting attacks due to the ability of recognizing various attack patterns [16]. As an example, clustering methods such as k-means were applied for classification of different types of attacks (e.g., DDoS attacks [6]).

Another common approach is the analysis of attackers' intentions, performed through behavior-based detection [9]. Such an approach helps reveal rather complex and multistep attack scenarios.

However, current detection approaches operate in an offline environment or semi-online mode, preventing them from responding to attacks in real time. Furthermore, most existing approaches target general IoT systems and do not consider the peculiarities of IP cameras in terms of attacks and attack behavior. The relationship between these attack types and the corresponding attacker activities is summarized in Table 1.

Table 1 Analysis of Attack Types and Attacker Activities

Type of Attack	Description	Attacker Activities
Brute Force	Multiple login attempts using different passwords	Authentication attacks
High Request Rate	Frequent requests in a short time	Scanning and reconnaissance
Unauthorized Access (/admin)	Access restricted pages	Privilege escalation
RTSP Vulnerability	Manipulation of video streams	Surveillance and spying
Malware Attack	Execution of malicious software	System compromise

These mappings illustrate that attacks can show the purpose behind an attacker's actions, thus proving the need for detection methods based on behavior.

3.4 Comparative Analysis of Existing Systems

Comparing the available research, we can identify some deficiencies in the existing IoT security systems. The majority of them rely on either attack detection or deception techniques, but very few combine the two approaches in one framework.

The examples of the former are the following: CamDecept and CamDec which create a realistic environment for the attacker, but their analytical skills are not highly developed [5], [11]. In turn, machine learning-based detection systems ensure higher accuracy but do not interact with attackers [16].

Honeypots like HoneyIoT and AIIPot have advanced attacker interactions and data collection abilities but are not optimized to be used in the environment of IP cameras [8], [10]. Also, SweetCam allows interaction with protocols but may lack realism [7].

Table 2 Comparison of Existing IoT and IP Camera Honeypot Systems

Study	IP Camera Support	AI Usage	Real-time Detection	Behavior Analysis	Main Attacks Covered	Limitations
SIPHON [1]	Yes	No	No	Yes	Brute force, malware	Complex deployment
IoT Candy Jar [2]	Yes	Yes	Partial	Yes	Brute force, botnet	Requires training
ThingPot [3]	No	No	No	Partial	Protocol attacks	Not camera specific
SweetCam [11]	Yes	No	No	Yes	Command injection	Limited realism
HoneyIoT [12]	No	Yes	Yes	Yes	Malware, botnet	Not IP camera specific
AIIPot [14]	No	Yes	No	Yes	Session-based attacks	No camera protocol support
CamDec [15]	Yes	No	No	Yes	RTSP, SSH	High complexity

From Table 2, it is clear that the current methods lack an integrated system that considers the factors of real-time detection, intelligent behavioral analysis, and other issues related to the IP camera attack scenario. In this regard, the proposed system will attempt to integrate the factors of deception techniques, real-time detection, and behavioral analysis of the attacker within the same system framework applicable to the IP camera environment.

3.5 Research Gap and Problem Alignment

From a review of existing literature, we identify several main issues in relation to the IoT security tools and measures proposed today.

Firstly, existing solutions do not specialize in IP cameras. There are many IoT systems available on the market, but very few target camera-related problems and vulnerabilities such as RTSP attacks and manipulating video feeds [4], [14].

Secondly, many of those solutions cannot offer real-time monitoring capabilities. Most of the analyzed studies either use offline or semi-real-time methods, thus limiting their applicability during the actual attack process.

Thirdly, the behavior of potential attackers is often not taken into consideration. Although a lot of research works concentrate on collecting attack data, they do not fully analyze attack behavior or the stages of the multi-phase attack process [9].

Fourthly, deception techniques are not linked with the analysis tools used today. Honeypot techniques can be employed for luring in attackers while artificial intelligence may help with detecting and analyzing those attacks.

Thus, The proposed system combining IP camera-based honeypot, real-time monitoring, and behavior analysis will address all of the aforementioned gaps.

Chapter 4

System Analysis

4.1 System Overview

The proposed system is an advanced cybersecurity solution based on a honeypot architecture, specifically designed to emulate the behavior, interfaces, and network characteristics of an IP camera within a controlled and isolated environment. This system falls under the domain of Internet of Things (IoT) security and aims to address the increasing number of cyber threats targeting smart surveillance devices.

The primary objective of this honeypot is to act as a decoy system that intentionally exposes simulated vulnerabilities in order to attract and engage malicious actors. By mimicking real-world IP camera functionalities—such as HTTP/HTTPS web interfaces, Real-Time Streaming Protocol (RTSP) services, and authentication mechanisms—the system enhances its realism and increases the probability of interaction from both automated attack tools (bots) and sophisticated human adversaries.

From a technical perspective, the system implements data acquisition and logging mechanisms to capture a wide range of attack-related information. This includes network traffic metadata, source IP addresses, geolocation indicators, timestamps, request methods, payload signatures, and authentication attempts. These data points are stored in a structured database to support further forensic analysis and threat intelligence generation.

The analytical component of the system leverages behavioral analysis techniques to identify attack patterns and classify different types of threats, such as brute force attacks, port scanning activities, credential stuffing, and exploitation of known vulnerabilities (e.g., CVEs related to IoT devices). This enables the extraction of actionable insights that can contribute to improving intrusion detection systems (IDS) and intrusion prevention systems (IPS).

To ensure operational security and risk mitigation, the honeypot is deployed within a sandboxed environment with strict network isolation policies. This containment strategy guarantees that any compromise of the honeypot does not propagate to production systems or sensitive infrastructure, thereby maintaining the integrity and confidentiality of real assets.

In addition, the system supports research-driven objectives by providing a platform for cybersecurity experimentation, attack simulation, and threat modeling. It aligns with modern cybersecurity practices such as proactive defense, attack surface analysis, and adversarial behavior monitoring.

4.2 Functional Requirements

The system needs to offer a suite of functionalities that will help in simulating the environment of an IP camera and detecting the attackers behavior. This involves emulating a number of camera services such as login through a web portal, RTSP stream, and SSH connections, among others, so that the attackers can interact.

In addition, the system must record all the actions taken by the attackers, including login actions, the services accessed, the commands executed, and the time stamps. Moreover, the system needs to have detection rules that help in identifying pre-defined attack patterns such as authentication brute-forcing, RTSP unauthorized access, and port scanning.

Moreover, the system needs to trigger alerts whenever suspicious activities take place and provide a centralized logging mechanism to store and analyze attack data for further investigation.

To ensure the effectiveness of the proposed CamDecept framework, a set of functional requirements was identified. These requirements define the main functionalities that the system should provide for simulating IP camera services, detecting attacks, analyzing attacker behavior, and generating alerts. Table 3 summarizes the key functional requirements of the proposed system.

ID	Functional Requirements
FR1	The system shall simulate IP camera services such as HTTP, RTSP, and SSH.
FR2	The system shall detect unauthorized login attempts in real time.
FR3	The system shall analyze attacker behavior patterns.
FR4	The system shall generate alerts for suspicious activities.
FR5	The system shall securely store attack logs and records.
FR6	The system shall support centralized monitoring and analysis

Table 3: Functional Requirements of the CamDecept System

4.3 Non-Functional Requirements

A number of non-functional requirements should be fulfilled in order to make the system effective and reliable. First of all, it should be lightweight and scalable, meaning that the system is going to run without the necessity to have special hardware.

The second requirement is that the system needs to guarantee a very high level of realism so that an attacker does not realize he or she is dealing with a honeypot. It means that the interface will be realistic and will show appropriate reactions.

Thirdly, security and isolation are important requirements because the honeypot should work in a closed environment in order not to enable an attacker to get to any other computer via the honeypot.

Finally, the honeypot should guarantee fast logging and a quick reaction on possible attacks.

ID	Non-Functional Requirements
NFR1	The system shall provide security and isolation to prevent attackers from accessing external systems.
NFR2	The system shall support lightweight deployment without requiring special hardware.
NFR3	The system shall provide a simple and usable interface for administrators.
NFR4	The system shall support scalability for handling multiple connections and future extensions.
NFR5	The system shall provide fast and efficient logging performance.
NFR6	The system shall protect the privacy of collected logs and stored data.
NFR7	The system shall support maintainability and future updates.

Table 4: Non-Functional Requirements of the CamDecept System

4.4 Abuse Cases Scenarios

The system evaluates a variety of abuse cases that reflect the types of attacks the attacker might use against IP camera services.

- **Brute Force Authentication Attack**

The attacker tries to gain access into the system by logging into the camera using different usernames and passwords on the camera login interface. This behavior is detected by the system through multiple failed log-in attempts, resulting in alerts.

- **Unauthenticated RTSP Stream Access Attempt**

The attacker tries to gain access to the video stream from the camera without being authenticated via RTSP requests. The system logs all RTSP access attempts also identifies abnormal or repeated **requests as suspicious activity**.

- **Port Scanning / Service Discovery**

The attacker performs scanning activities to detect opened ports and available services including HTTP, RTSP, and SSH. The system detects unusual scanning patterns also correlates multiple requests to identify reconnaissance behavior.

- **SSH Attack Interaction**

The attacker accesses the system through SSH and performs actions within it. The honeypot records all commands and interactions, enabling analysis of attacker techniques and behavior.

As shown in Figure 2, the flow starts involved in this attack begin when the attacker interacts with the IP camera honeypot system which emulates various services such as HTTP, RTSP, and SSH to detect any brute force attack attempts, access unauthorized to live stream as well as command execution. All these actions are sent to the central logging system by the detection engine for further analysis to generate alerts or responses.

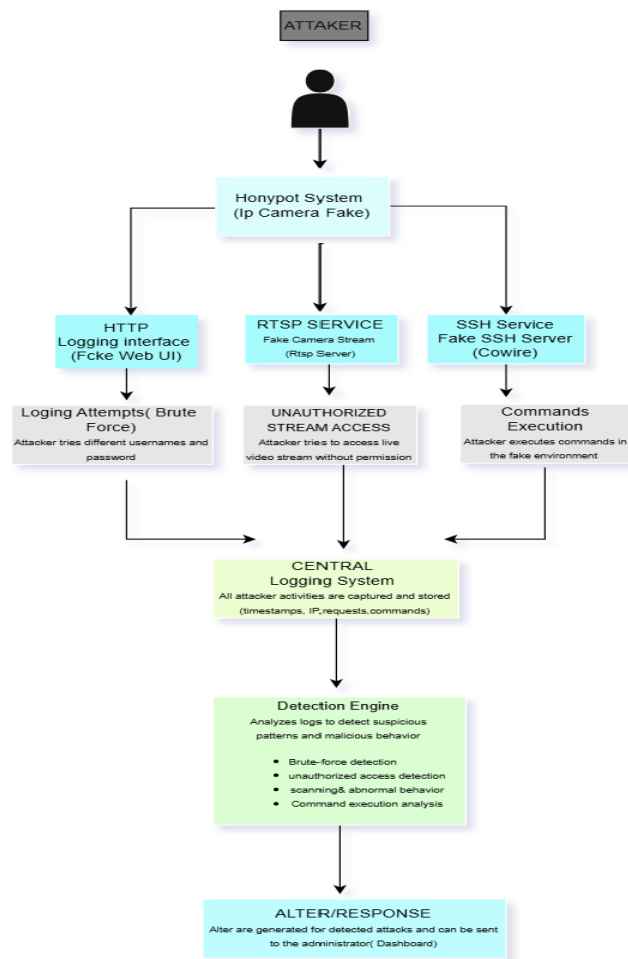


Figure 2 IP Camera Honeypot Attack Flow and Detection Process

As Shown in Figure 3, the abuse scenario diagram depicts the major attack scenarios targeting on the honeypot framework for IP camera. The attacks include HTTP brute force attacks, RTSP unauthorized access, default credentials attack, and ONVIF discovery scan attack. The responses from the honeypot framework to such attacks include detection, logging, alerting, and log storage.

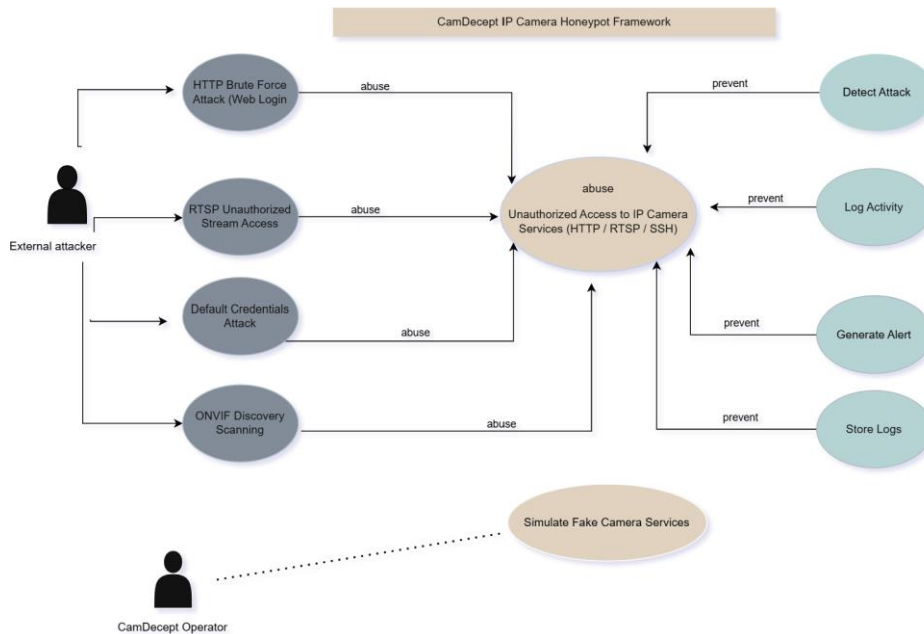


Figure 3 Abuse Case Diagram of the CamDecept Framework

As Shown in Figure 4, shows the case diagram for attack and defense abuse that depicts the interactions between the attacker's activities and the system defense mechanisms. The attacker may use brute force attacks, credential stuffing attacks, and protocol-based attacks, whereas the framework will monitor, classify, issue alerts, and store information safely to minimize unauthorized access risks.

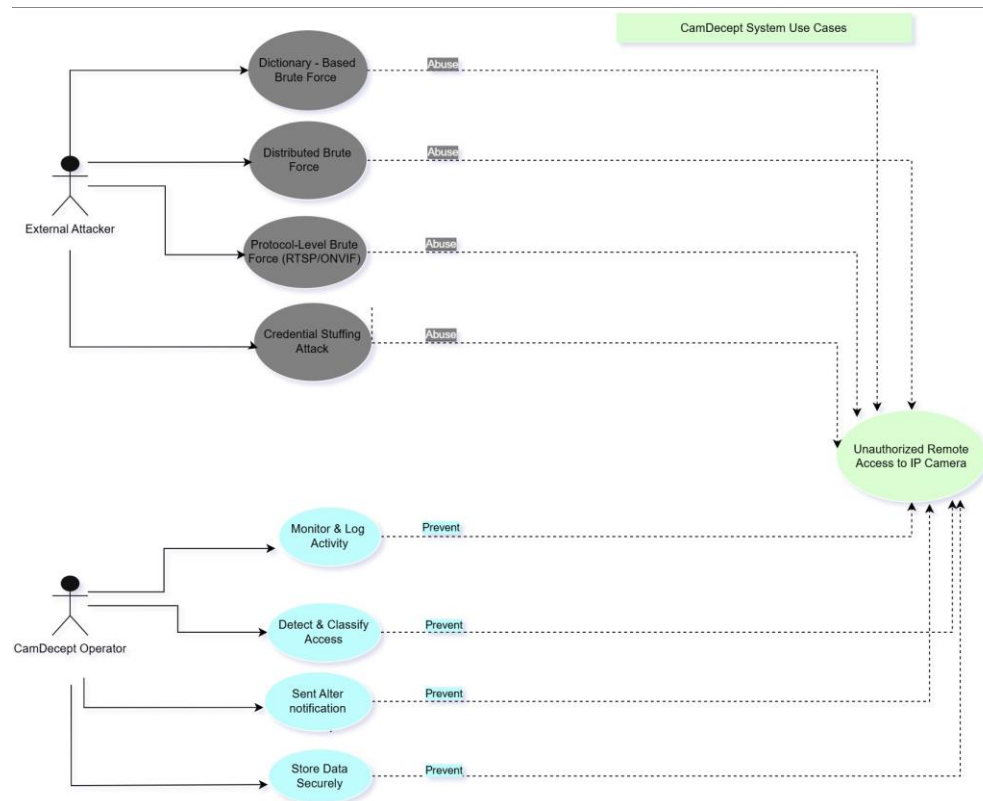


Figure 4 Attack and Defense Diagram of CamDecept

4.5 Therat Model

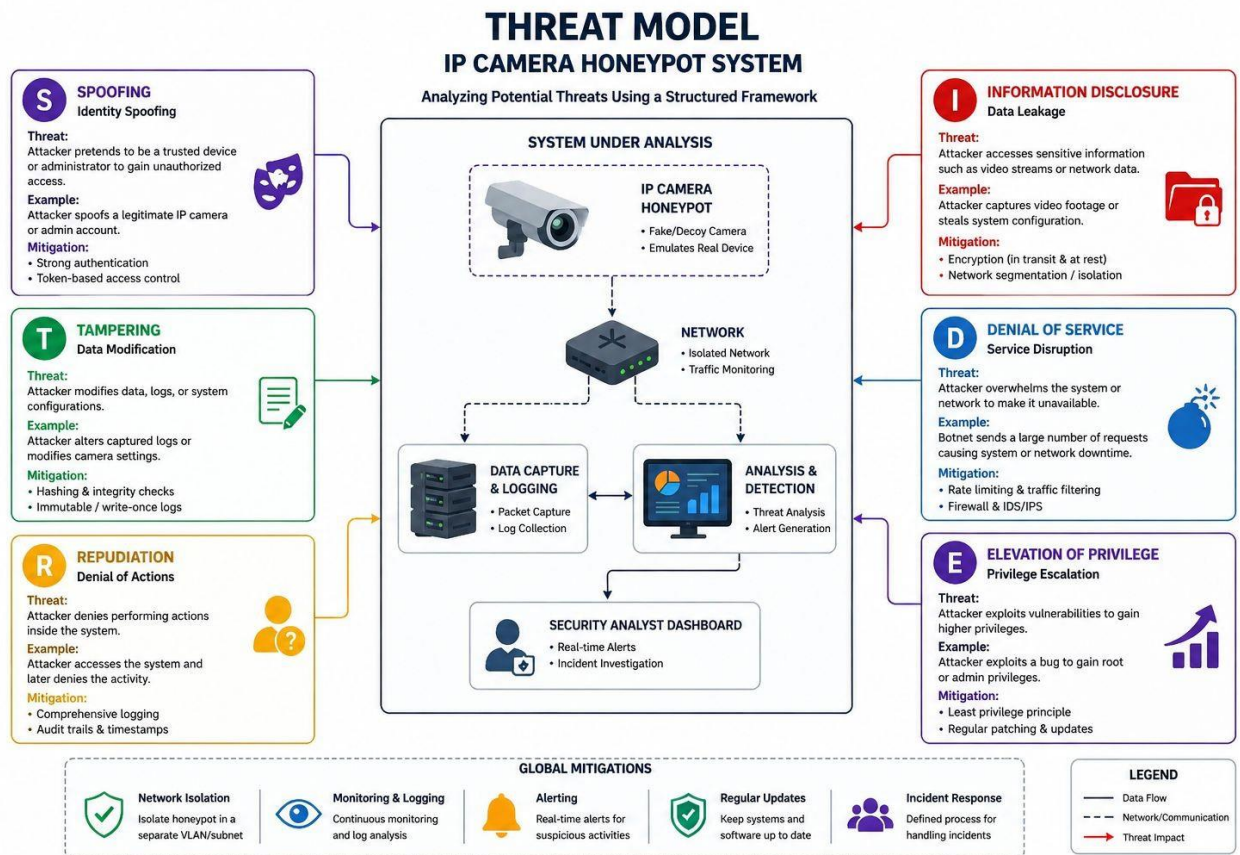


Figure 5 shows a threat model for an IP Camera Honeypot system, including its components, possible attacks, and security protections.

Threat Model Description for IP Camera Honeypot System

This figure illustrates a threat model for an IP Camera Honeypot System. The purpose of this model is to identify and analyze potential security threats targeting a decoy IP camera designed to attract attackers and monitor their behavior. Unlike a real camera, the honeypot simulates a vulnerable device to capture malicious activities without exposing the actual network.

The system includes a Data Capture and Logging component, where all incoming traffic, login attempts, and attacker actions are recorded. These logs are then processed by the Analysis and Detection module, which identifies suspicious patterns and generates alerts. The results are displayed on a Security Analyst Dashboard, allowing security teams to monitor incidents in real time and perform further investigation.

Several categories of threats are identified in this model. Spoofing refers to attackers impersonating a trusted device or administrator to gain unauthorized access. Tampering involves modifying system data, logs, or configurations to hide malicious activities. Repudiation occurs when an attacker denies performing actions within the system, making proper logging and timestamps essential.

Information Disclosure refers to unauthorized access to sensitive data such as logs or system configurations. Denial of Service (DoS) occurs when attackers overwhelm the system with excessive requests, causing service disruption. Elevation of Privilege happens when attackers exploit vulnerabilities to gain higher-level access, such as administrative privileges.

To mitigate these threats, multiple security controls are implemented. These include strong authentication mechanisms, data integrity checks, encryption, network isolation, traffic filtering, and regular system updates. Additionally, global mitigation strategies such as continuous monitoring, real-time alerting, and incident response planning enhance the overall security posture of the system.

This threat model demonstrates a layered security approach, where the honeypot is intentionally exposed to attackers while remaining isolated and monitored. This allows organizations to safely collect threat intelligence and improve their defensive capabilities without risking the integrity of the real network.

4.5 System Architecture Overview

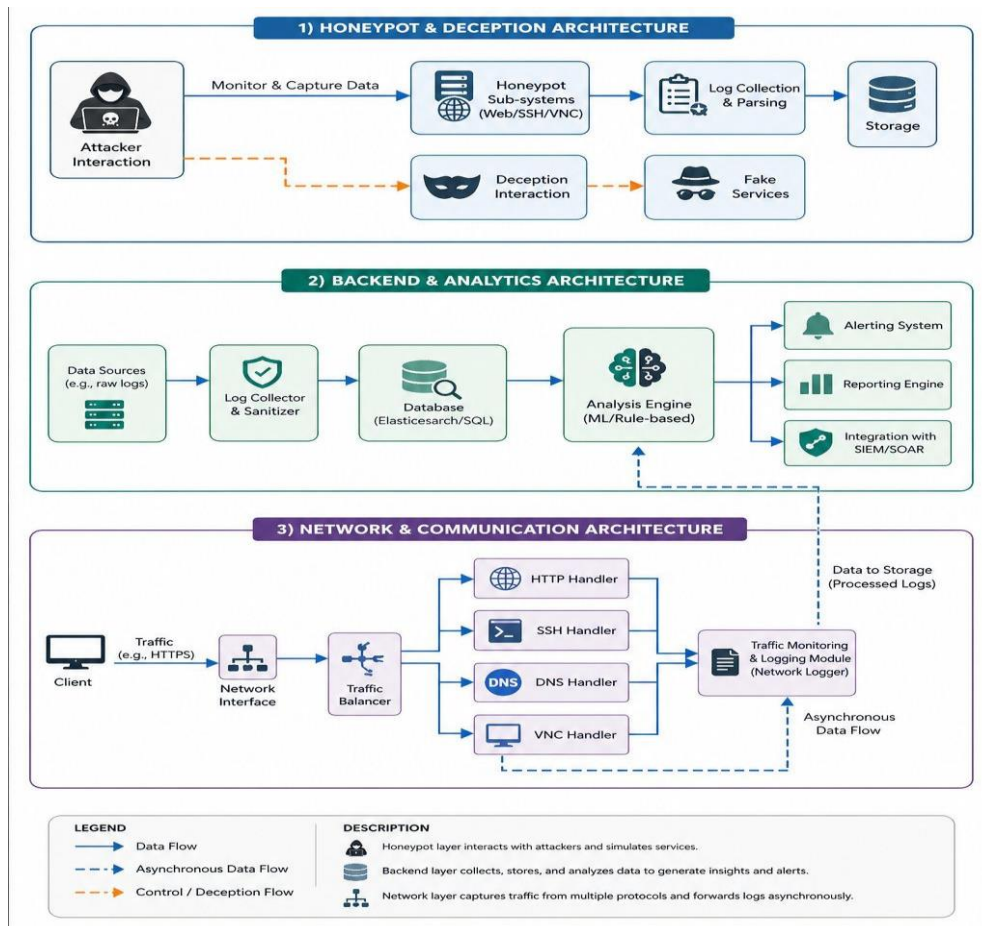


Figure 6 Network and Communication Architecture

The presented diagram illustrates a multi-layered cybersecurity architecture that integrates honeypot technology, network monitoring, and backend analytics to detect, analyze, and respond to cyber threats targeting IP-based systems. The architecture is designed using a modular approach and is divided into three main layers.

1. Honeypot and Deception Architecture

This layer represents the primary interaction point between external attackers and the system.

Components:

- Attacker Interaction Represents external entities attempting to access or exploit the system.
- Honeypot Subsystems (Web, SSH, VNC) These components simulate real services in order to attract attackers and capture their behavior without exposing actual assets.
- Log Collection and Parsing Responsible for capturing and structuring attacker activity into organized log data.
- Storage Stores the collected data for further analysis.
- Deception Interaction and Fake Services Provides controlled, simulated responses to attackers in order to prolong engagement and gather deeper behavioral insights.

Purpose:

This layer is designed to lure attackers, monitor malicious activities, and safely collect attack data.

2. Backend and Analytics Architecture

This layer is responsible for processing, analyzing, and transforming collected data into actionable intelligence.

Components:

- Data Sources (Raw Logs) Includes logs generated from the honeypot layer and network monitoring components.
- Log Collector and Sanitizer Cleans and normalizes log data to ensure consistency and reliability.
- Database (e.g., Elasticsearch or SQL) Stores structured data to support efficient querying and analysis.
- Analysis Engine (Machine Learning or Rule-Based) Acts as the core intelligence unit by detecting attack patterns, identifying anomalies, and analyzing behavior using predefined rules or machine learning techniques.
- Alerting System Generates alerts when suspicious or malicious activity is detected.
- Reporting Engine Produces analytical reports for security monitoring and decision-making.
- Integration with SIEM/SOAR Enables integration with enterprise security platforms to support automated response and orchestration

Purpose:

This layer transforms raw data into meaningful insights and supports effective security decision-making and incident response.

3. Network and Communication Architecture

This layer handles traffic intake, distribution, and logging across multiple network protocols.

Components:

- Client Represents the source of incoming traffic, which may be legitimate or malicious.
- Network Interface Serves as the entry point for all incoming network traffic.
- Traffic Balancer Distributes traffic across multiple handlers to simulate a realistic system environment.
- Protocol Handlers (HTTP, SSH, DNS, VNC) Each handler processes interactions specific to its protocol.
- Traffic Monitoring and Logging Module (Network Logger) Collects and aggregates traffic data from all protocol handlers.

Data Flow Explanation

The system operates using a hybrid data flow model:

- Direct Data Flow

Traffic flows from the client to protocol handlers and then to the network logger. Similarly, honeypot data flows directly to log collection and storage components.

- Asynchronous Data Flow

The network logger forwards processed log data asynchronously to the backend components, including the log collector, database, and analysis engine. This background processing approach improves performance, reduces latency, and enhances scalability.

Chapter 5

Implementation

5 Implementation

This chapter presents the process of development of CamDecept, the IoT Camera Deception Framework which represents one of the main achievements of this work. It includes the architecture of the whole system, details of its components, the mechanism of detection, the process of calculating the risk score, as well as a description of the loggers and the alerting system with relevant code snippets and output screenshots from the actual implementation.

The proposed system relies on two Python modules: `cam_2.py` is a honeypot server built using Flask which creates a realistic fake web server of an IP camera exposing several HTTP endpoints. The second module, called `ai_analyzer.py`, is a behavior analysis engine which receives requests for analysis and calculates the risk score based on the detection rules. These two modules form an integrated pipeline from capturing attacker requests to behavior classification and producing alerts.

CamDecept was developed progressively and tested using various attack simulations which consisted of three types: reconnaissance and scan attempts, unauthorized access to both admin and stream endpoints, and authentication attacks. The results of the above-mentioned tests are reported in this chapter.

5.1 System Architecture Design

CamDecept aims to be a modular and three-tier deception solution, designed to lure, detect, and study cyber-attacks targeting IP camera systems. The architecture separates areas of interest into three highly interrelated tiers: the Deception Service Tier, the Intelligence and Detection Tier, and the Presentation and Alerting Tier. In this way, each tier can be developed and enhanced independently but still maintain clear and consistent data flows from attacker interaction through to dashboard display.

The high-level design concept reflects the idea of minimum footprint and maximum observability, as outlined in honeypot design considerations presented by Peradilla et al. [6] in their "IP Camera Honeypot DoS Analysis." The idea is to have a light-weight

honeypot service for efficient deployment, while the detection and logging modules should be fully observable.

5.1.1 Three-Layer Architecture

CamDecept is designed as a three-tier modular deception framework based on being able to both attract and then capture, as well as analyze cyberattacks against IP camera infrastructures. Each of the three types of deceptions is accomplished through very loosely integrated processes that are highly integrated across three functional layers. The function layers are: Deception Service Layer, Intelligence and Detection Layer, and Presentation & Alerting Layer. This separation of concerns across these three layered functions allows each of the functional components to be developed, tested and expanded upon independently, while still being able to function together in achieving a cohesive end to end data flow from the time the attacker first interacts with the deception to the subsequent viewing of the results in a dashboard.

Table 3 CamDecept three-layer architecture

Layer	Module	Responsibility
Deception Service	cam_2.py (Flask)	Expose fake IP camera endpoints, capture raw HTTP interactions
Intelligence & Detection	ai_analyzer.py	Behavioral feature extraction, risk scoring, attack classification
Presentation & Alerting	Dashboard (localhost:8501)	Real-time visualization, log browsing, alert display

The two primary Python modules, namely cam_2.py as well as ai_analyzer.py, make up the heart of this system. Cam_2.py acts as the honeypot server (using Flask) by receiving and processing all HTTP requests coming in, while ai_analyze.py gives the system its behavioral analysis engine that considers the entire history of a specific attacker,

analyzing every single request from that attacker through their total session history. The two modules are connected by an easy-to-use function call interface, in that `after_request` hook from `cam_2.py` calls `ai_analyzer.analyze_event()` for every HTTP request that was processed (in terms of being complete), passing it the entire request context, and receiving back a structured set of analyzed data.

5.1.2 Data Flow

An end-to-end data flow through the CamDecept architecture occurs via:

- 1- An attacker or automated scanning tool will generate an HTTP request addressed to one of the exposed honeypots request interfaces(s) (e.g., GET /admin, POST /login, GET large file).
- 2- `cam_2.py` will use its `before_request` hook and retrieve the source IP, request path, HTTP method, and User-Agent string, and store these attributes in the request context object (`_ihdf_meta`).
- 3- Once `cam_2.py` has finished processing the request, it will route the request to the appropriate endpoint handler, retrieving a response that is deceptively similar, but ultimately misleading from what they were attempting (i.e., HTTP Code 403 - Admin will not be granted, HTTP Code 401 - User cannot authenticate to access Login and JSON payload will be sent back for /status checks) and then return it to the original source.
- 4- The `after_request` hook will call `ai_analyzer.analyze_event()` passing the entire set of request metadata captured for the request along with the HTTP response status code returned.
- 5- `ai-analyzer` tracks individual IP addresses' event data using a sliding window (60 seconds) and extracts features that describe the IP's behaviour, and calculates a cumulative risk score for those events, then returns a structured dictionary of those results.

6- cam-2.py writes the full event to logs.jsonl, and when an event has a risk level of MEDIUM or HIGH, cam-2.py also writes a structured alert to alerts.jsonl for that same event.

7- If an event has a risk level of HIGH, cam-2.py will inject an X-IHDF-Response: a deeper-decoy-engaged response header with the HTTP response to indicate that a deeper deception has been initiated.

8- The dashboard reads the log and alert files to display the entries as real-time visualizations and as searchable tabular forms of the recorded entries.

5.2 Components Description

The CamDecept framework consists of five core components that work together to form a complete deception and detection pipeline. The following table provides an overview of each component, its purpose, and the data it receives and produces.

Module/File	Purpose	Input	Output
cam_2.py	Honeypot Flask server	HTTP requests	Fake responses + logs
Ai_analyzer.py	Behavior analysis	Request metadata	Attack type + risk score
Logs.jsonl	Stores all events	Request records	Full activity history
Alerts.jsonl	Stores alerts	Medium/High events	Alert records
Dashboard	Visualization	Logs/alerts	Charts and tables

Table CamDecept system components overview

5.2.1 cam_2.py - Honeypot Flask Server

cam_2.py is the primary entry point of the CamDecept framework, implemented using Flask - a lightweight Python WSGI web framework. Following the approach of SweetCam [7], which demonstrated that a multi-protocol honeypot server can capture diverse IP camera attack patterns, cam_2.py exposes five decoy endpoints that collectively simulate the attack surface of a real IP camera device:

```
"source_ip": "127.0.0.1", "path": "/status", "method": "GET", "status": 200}
"source_ip": "127.0.0.1", "path": "/favicon.ico", "method": "GET", "status": 404}
"source_ip": "127.0.0.1", "path": "/status", "method": "GET", "status": 200}
"source_ip": "127.0.0.1", "path": "/login", "method": "GET", "status": 405}
"source_ip": "127.0.0.1", "path": "/status", "method": "GET", "status": 200}
"source_ip": "127.0.0.1", "path": "/admin", "method": "GET", "status": 403}
"source_ip": "127.0.0.1", "path": "/stream", "method": "GET", "status": 200}
"source_ip": "127.0.0.1", "path": "/admin", "method": "GET", "status": 403}
"source_ip": "127.0.0.1", "path": "/admin", "method": "GET", "status": 403}
```

Figure 7 Live HTTP responses captured

The logged responses from all the various decoy endpoints confirm that all endpoints are returning appropriate HTTP status codes for all requests that they received, which matches the behavior of an actual IP camera. An example of this is that when a user makes a GET request to the /login endpoint, a 405 is returned as this endpoint is only configured to accept POST requests. This specific endpoint has been specifically designed to match the behavior of an actual IP camera web interface and prevents enumerating the login surface through trivial means. The /admin endpoint always returns 403 to the user to simulate a real access control boundary, causing an attacker to escalate their efforts, and thereby providing additional behavioral telemetry for the detection engine's use.

5.2.2 ai_analyzer.py — Behavioral Analysis Engine

ai_analyzer.py is an analyser that uses a rule-based behavioral detection engine with a risk scoring model. It provides the behavioral core of CamDecept through a stateful sliding window of HTTP events maintained on a per-IP basis for incoming HTTP events. Each event is evaluated against a multi-dimensional feature extraction and scoring model. This is achieved using the Adaptive Interaction Modeling approach from AIIPot [10],

which provides a mechanism for preserving attacker session context across requests and thus allows for detecting behavioral patterns.

The module employs two data structures from the Python collections library: defaultdict and deque. defaultdict maps each source IP address to a deque of event records, supporting $O(1)$ insertions of events and $O(k)$ pruning of expired events (where k is the number of events that have a timestamp older than the 60-second window boundary). This design results in an analyzer memory footprint that will scale with the number of active attacker sessions, not the total number of events, enabling long-term deployment in high-volume attack scenarios as documented by Tabari et al. [9].

5.3 Detection Mechanism

The CamDecept detection mechanism is implemented entirely within ai_analyzer.py as a multi-rule behavioral classifier. Unlike signature-based intrusion detection systems, CamDecept's approach is session-aware: it evaluates each request in the context of the attacker's complete behavioral history within the current 60-second sliding window. This enables the system to distinguish between isolated probes and sustained attack campaigns a distinction highlighted as critical by Alrashdi et al. [16] in their review of ML-based intrusion detection systems for IoT applications.

5.3.1 Sliding Window Event Management

Each inbound request is appended to a per-IP deque with a datetime timestamp. Before feature extraction begins, the prune_old() function removes all events outside the 60-second window:

```

WINDOW_SECONDS = 60

events = defaultdict(lambda: deque())

SENSITIVE_PATHS = {"/admin", "/config", "/control", "/login"}
STREAM_PATHS = {"/stream", "/live", "/snapshot"}

def prune_old(ip):
    cutoff = datetime.utcnow() - timedelta(seconds=WINDOW_SECONDS)
    dq = events[ip]

    while dq and dq[0]["ts_dt"] < cutoff:
        dq.popleft()

```

Figure 8 ai_analyzer.py initialization

The 60-second window was selected to balance detection sensitivity against false positive resistance — short enough to capture rapid automated attack sequences (as observed in the DoS patterns documented by Peradilla et al. [6]), yet long enough to avoid misclassifying slow, low-volume probes as high-urgency attacks.

5.3.3 Detection Rules

Six detection rules are evaluated sequentially against the extracted feature vector:

```

if total_requests > 20:
    score += 3
    attack_type = "Reconnaissance / High Request Rate"
    indicators.append("High request rate")

if unique_paths > 8:
    score += 2
    attack_type = "Scanning Behavior"
    indicators.append("Multiple endpoints accessed")

if failed_auth > 5:
    score += 4
    attack_type = "Brute Force / Failed Authentication"
    indicators.append("Multiple failed authentication attempts")

```

Figure 9 Detection rules R1, R2, and R3 with their score contributions.

First three detection rules in ai_analyzer.py: R1 flags high request rate (>20 requests, +3), R2 identifies scanning behavior (>8 unique paths, +2), and R3 detects brute-force attempts (>5 failed authentications, +4).

```
if sensitive_hits > 0:
    score += 3
    attack_type = "Unauthorized Access Attempt"
    indicators.append("Sensitive endpoint accessed")

if stream_hits > 0:
    score += 3
    attack_type = "Unauthorized Stream Access"
    indicators.append("Camera stream endpoint accessed")

if user_agent and any(tool in user_agent.lower() for tool in ["curl", "powershell", "python", "nmap", "zgrab"]):
    score += 2
    indicators.append("Suspicious user-agent detected")
```

Figure 10 Detection rules R4, R5, and R6 with their score contributions.

Remaining detection rules: R4 triggers sensitive endpoint access (+3), R5 on camera stream access (+3), and R6 adds +2 for known attack tool User-Agents (curl, powershell, python, nmap, zgrab) without overriding the attack_type label.

5.4 Risk Scoring Model

The risk scoring model translates the multi-dimensional behavioral feature vector into a single cumulative risk score and a categorical risk level. This design is theoretically grounded in the Markov Decision Process (MDP) framework [17], where attacker-honeypot interactions are modeled as sequences of state transitions an approach formally developed by Guan et al. [12] in their HoneyIoT runtime adaptation work using reinforcement learning. In CamDecept's adaptation, each HTTP request constitutes a state transition whose cumulative weight reflects the attacker's observed behavioral pattern:

$$T(s, a, s') = P(S_t = s' \mid S_{t-1} = s, a_t = a) \\ = \frac{C(s, a, s')}{\sum_{x \in S} C(s, a, x)}$$

Figure 11 MDP state transition probability formula

Where $C(s, a, s')$ represents the count of observed transitions from state s to state s' following action a .

```

if score >= 7:
    level = "HIGH"
    action = "Fake response + alert + isolate to decoy"
elif score >= 3:
    level = "MEDIUM"
    action = "Alert operator"
else:
    level = "LOW"
    action = "Log only"

```

Table 4 Risk level

Risk Level	Score Range	Interpretation	Recommended Action
LOW	0 – 2	Normal or marginally anomalous	Log only
MEDIUM	3 – 6	Potential attack, warrants attention	Alert operator, write to alerts.jsonl
HIGH	$7 \geq$	Active malicious behavior	Fake response + alert + isolate to decoy

The HIGH risk threshold of 7 was calibrated empirically during testing. A single attacker triggering R4 (+3) and R5 (+3) with a PowerShell User-Agent R6 (+2) reaches a score of 8 immediately, ensuring that multi-vector attacks are escalated without delay. This graduated response strategy aligns with the deception architecture described by Tabari et al. [14], who demonstrated that adaptive honeypot responses based on attacker behavioral classification significantly improve engagement duration and intelligence yield.

5.4.1 Score Progression Analysis

Table 5 presents the score progression observed across the five simulated attack scenarios, showing how different combinations of triggered rules produce varying peak scores and final risk classifications.

Table 5 Score progression across simulated attack scenarios

Attack Scenario	Requests	Rules Triggered	Peak Score	Risk Level
Browser recon	5	R4 only	3	MEDIUM
PowerShell /admin loop	20	R1 + R4 + R6	8	HIGH
PowerShell /admin loop	25	R1 + R4 + R6	8	HIGH
PowerShell /stream + /admin	+30	R1 + R4 + R5 + R6	11	HIGH
Multi-path scan (>8 endpoints)	15	R2 + R4 + R6	7	HIGH

5.5 Implementation Details

This section presents the practical implementation details of CamDecept, covering the development environment used during the project and the integration between the two core modules. These details provide the technical context necessary to understand how the system was built, deployed, and tested.

5.5.1 Development Environment

The CamDecept application was implemented and air-tested utilizing Windows 10 (Build 10.0.26200) as the environment, built upon version 3.x of Python, and version 2.x

of Flask in debug mode with the application hosted locally on port 5000, while users can access the dashboard on port 8501. All attacks were performed on the same machine where multiple PowerShell scripts as well as browser-based requests simulating real attacks were executed for the purpose of simulating real-life attacks in a controlled environment.

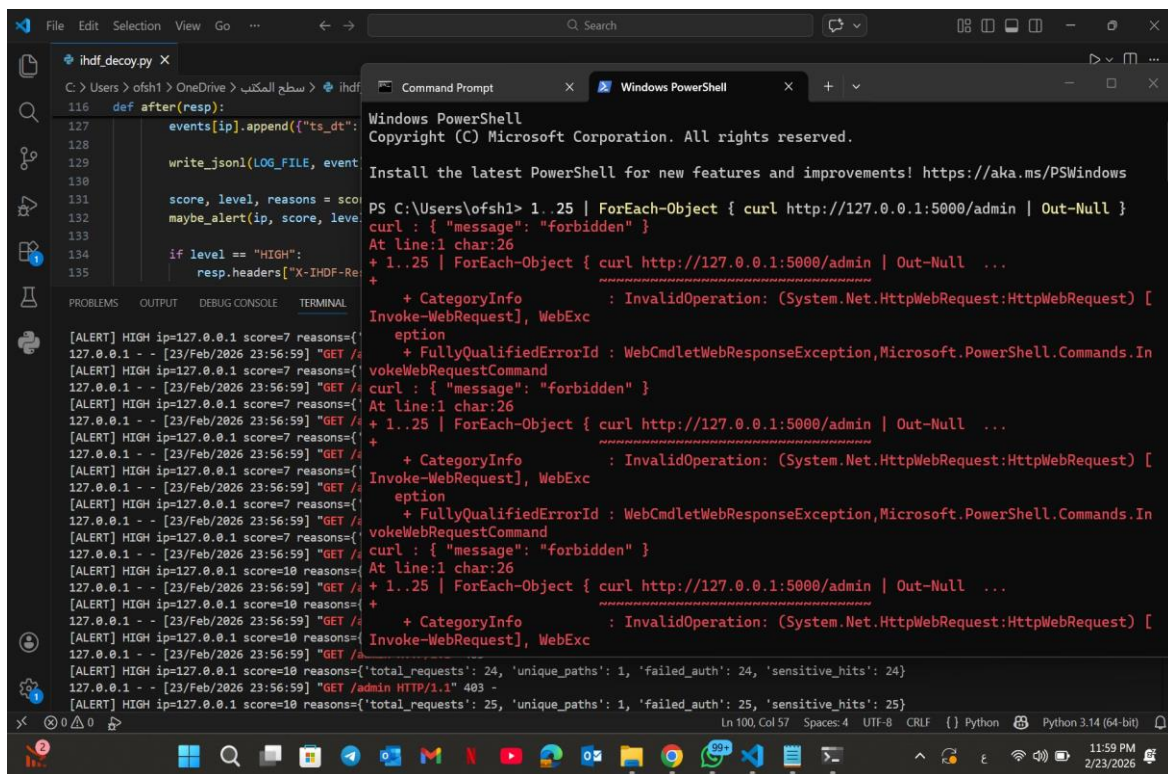


Figure 12 Live attack simulation

Using the command `1..25 | ForEach-Object { curl http://127.0.0.1:5000/admin | Out-Null }`, for example, would send 25 HTTP GET requests in sequence to the `/admin` endpoint very quickly; these requests would all receive a "403 forbidden" response from the honeypot, showing that the attacker is not able to see if they are being monitored or judged as a result of their actions. This is an accurate representation of the actions that automated attacking tools would perform during lengthy automated probes against endpoint URLs looking for possible misconfiguration (using a URL for which they are unauthorized to have access) or authentication bypass vulnerabilities — a behavior we have seen repeatedly in the deployment of honeypot IP cameras as noted by Tabari et al. [9].

Parameter	Value
Operating System	Windows 10 (10.0.26200)
Language	Python 3.x
Framework	Flask 2.x (debug mode)
Honeypot Port	5000
Dashboard Port	8501
Log Format	JSONL (logs.jsonl / alerts.jsonl)
Attack Simulation Tools	PowerShell / Browser

Figure 13 CamDecept development and testing environment specifications.

All testing was conducted within this local environment to ensure full control over attack timing, volume, and source IP enabling precise validation of the detection engine's scoring thresholds without interference from external network traffic.

5.5.2 Module Integration

The integration of `cam_2.py` to `ai_analyzer.py` happens in a way. When `cam_2.py` gets a HTTP request, it calls the `analyze_event` function from `ai_analyzer`. The results from `analyze_event` are then used by `cam_2.py` to figure out how to log the event send alerts and change the response headers. All of this happens in one step, so it does not slow things down from the user's perspective.

This way every request is looked at. When `cam_2.py` responds to an IP address, it knows the latest risk score for that IP address. So `cam_2.py` can look at what's happening in real time without waiting. The integration of `cam_2.py` to `ai_analyzer.py` is good because it makes sure that every request is analyzed and that the risk score for an IP address is always up, to date when `cam_2.py` responds to it.

5.6 Logging and Monitoring

5.6.1 Request Log (logs.jsonl)

[illegible]

Figure 14 logs.jsonl entries showing browser reconnaissance transitioning to PowerShell attack traffic.

Graduation Project Report

5.6.2 Alert Log (alerts.jsonl)

Alert log entries are written only when analysis classifies an event as MEDIUM or HIGH risk. Alert records include the full behavioral feature vector, enabling retrospective analysis of the attacker's session state now each alert was generated — a capability aligned with the forensic logging architecture used in HoneyCam [13]

```
}

write_jsonl(LOG_FILE, event)

if analysis["level"] in ("MEDIUM", "HIGH"):
    alert = {
        "ts": now_iso(),
        "source_ip": ip,
        "score": analysis["score"],
        "level": analysis["level"],
        "attack_type": analysis["attack_type"],
        "indicators": analysis["indicators"],
        "features": analysis["features"],
        "action": analysis["recommended_action"]
    }

    write_jsonl(ALERT_FILE, alert)
```

Figure 15 Dual-log write logic: LOG_FILE for all requests, ALERT_FILE for MEDIUM/HIGH only

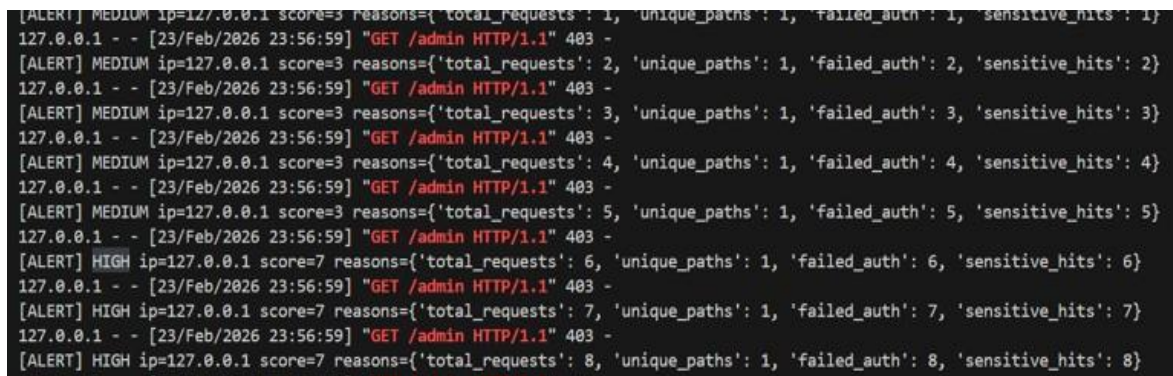
The two log files have different roles based on the code shown above. The unconditional LOG_FILE write gives forensic coverage to all honeypot interactions, including low-risk interactions. The conditional ALERT_FILE write gives a filtered source for intelligence, which includes only events that cross over a certain behavior threshold and therefore require analyst follow-up. This separation helps avoid alert fatigue by limiting the alert log records to only suspected activities and still allows retrospective investigations of lower-risk events that may become significant through time.

5.7 Alert Generation

The alert generation subsystem is the operational output of the CamDecept detection pipeline. It provides real-time notification of suspicious and malicious events through two parallel channels: structured file-based alerts (alerts.jsonl) for machine-readable downstream processing, and dashboard visualizations for human analyst consumption a dual-channel design consistent with the monitoring architecture proposed by Mfogo et al. [10] in AIIPot.

5.7.1 Alert Trigger Logic

Alerts are generated within the `after_request` hook of `cam_2.py`. Any event classified as MEDIUM or HIGH-risk results in an alert record being written to `alerts.jsonl`. For HIGH-risk events, the system additionally injects the X-IHDF-Response: deeper-decoy-engaged HTTP response header a mechanism aligned with the layered deception architecture of HoneyIoT [8]. Console alert output provides immediate visibility for operators monitoring the Flask terminal:



```
[ALERT] MEDIUM ip=127.0.0.1 score=3 reasons={'total_requests': 1, 'unique_paths': 1, 'failed_auth': 1, 'sensitive_hits': 1}
127.0.0.1 - - [23/Feb/2026 23:56:59] "GET /admin HTTP/1.1" 403 -
[ALERT] MEDIUM ip=127.0.0.1 score=3 reasons={'total_requests': 2, 'unique_paths': 1, 'failed_auth': 2, 'sensitive_hits': 2}
127.0.0.1 - - [23/Feb/2026 23:56:59] "GET /admin HTTP/1.1" 403 -
[ALERT] MEDIUM ip=127.0.0.1 score=3 reasons={'total_requests': 3, 'unique_paths': 1, 'failed_auth': 3, 'sensitive_hits': 3}
127.0.0.1 - - [23/Feb/2026 23:56:59] "GET /admin HTTP/1.1" 403 -
[ALERT] MEDIUM ip=127.0.0.1 score=3 reasons={'total_requests': 4, 'unique_paths': 1, 'failed_auth': 4, 'sensitive_hits': 4}
127.0.0.1 - - [23/Feb/2026 23:56:59] "GET /admin HTTP/1.1" 403 -
[ALERT] MEDIUM ip=127.0.0.1 score=3 reasons={'total_requests': 5, 'unique_paths': 1, 'failed_auth': 5, 'sensitive_hits': 5}
127.0.0.1 - - [23/Feb/2026 23:56:59] "GET /admin HTTP/1.1" 403 -
[ALERT] HIGH ip=127.0.0.1 score=7 reasons={'total_requests': 6, 'unique_paths': 1, 'failed_auth': 6, 'sensitive_hits': 6}
127.0.0.1 - - [23/Feb/2026 23:56:59] "GET /admin HTTP/1.1" 403 -
[ALERT] HIGH ip=127.0.0.1 score=7 reasons={'total_requests': 7, 'unique_paths': 1, 'failed_auth': 7, 'sensitive_hits': 7}
127.0.0.1 - - [23/Feb/2026 23:56:59] "GET /admin HTTP/1.1" 403 -
[ALERT] HIGH ip=127.0.0.1 score=7 reasons={'total_requests': 8, 'unique_paths': 1, 'failed_auth': 8, 'sensitive_hits': 8}
```

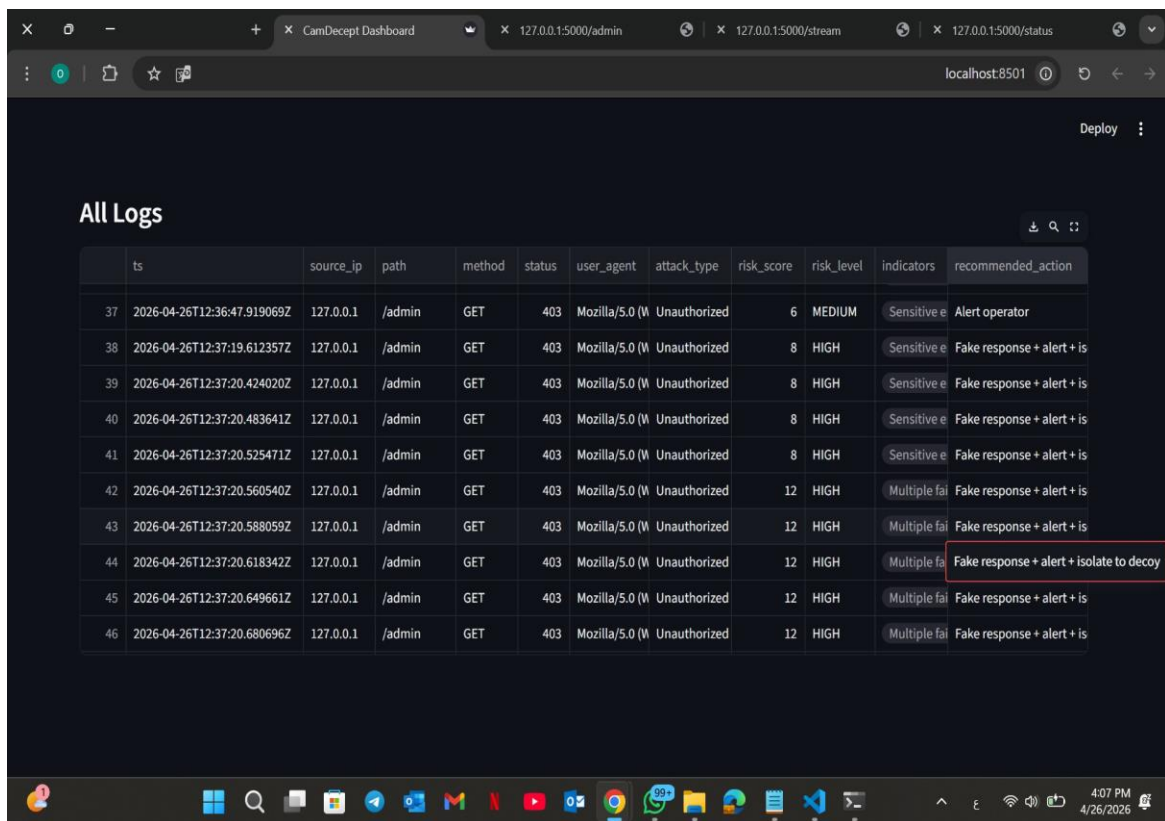
Figure 16 Flask terminal

The terminal output shows how the scoring model progresses. The first 5 requests are evaluated at a static score of 3 because of the number of `sensitive_hits` increasing with each `/admin` access. When the 6th request is processed, the `failed_auth` counter becomes greater than 5. This activates rule R3 (+4) as well as R4 (+3), causing an overall score of 7 to be reached, which crosses into the HIGH threshold. All future requests will be looked at as being HIGH risk and will cause the deeper-decoy-engaged response header to return to the client; thus, proving that the detection engine is able to successfully detect a change from opportunistic probing to confirmed malicious activity.

5.7.2 Dashboard Visualization

The CamDecept security dashboard (localhost:8501) provides three primary visualization components for analyst consumption.

Figure 17 presents the All-Logs table view, displaying enriched event records parsed from logs.jsonl. The table exposes all key fields including timestamp, source IP, path, HTTP method, status, User-Agent, attack_type, risk_score, risk_level, indicators, and recommended_action. The progressive escalation from MEDIUM (score 6, row 37) to HIGH (score 8+, rows 39 onwards) is clearly visible in the risk columns, consistent with the behavioral escalation patterns documented by Tabari et al. [14].



	ts	source_ip	path	method	status	user_agent	attack_type	risk_score	risk_level	indicators	recommended_action
37	2026-04-26T12:36:47.919069Z	127.0.0.1	/admin	GET	403	Mozilla/5.0 (W	Unauthorized	6	MEDIUM	Sensitive e	Alert operator
38	2026-04-26T12:37:19.612357Z	127.0.0.1	/admin	GET	403	Mozilla/5.0 (W	Unauthorized	8	HIGH	Sensitive e	Fake response + alert + is
39	2026-04-26T12:37:20.424020Z	127.0.0.1	/admin	GET	403	Mozilla/5.0 (W	Unauthorized	8	HIGH	Sensitive e	Fake response + alert + is
40	2026-04-26T12:37:20.483641Z	127.0.0.1	/admin	GET	403	Mozilla/5.0 (W	Unauthorized	8	HIGH	Sensitive e	Fake response + alert + is
41	2026-04-26T12:37:20.525471Z	127.0.0.1	/admin	GET	403	Mozilla/5.0 (W	Unauthorized	8	HIGH	Sensitive e	Fake response + alert + is
42	2026-04-26T12:37:20.560540Z	127.0.0.1	/admin	GET	403	Mozilla/5.0 (W	Unauthorized	12	HIGH	Multiple fai	Fake response + alert + is
43	2026-04-26T12:37:20.588059Z	127.0.0.1	/admin	GET	403	Mozilla/5.0 (W	Unauthorized	12	HIGH	Multiple fai	Fake response + alert + is
44	2026-04-26T12:37:20.618342Z	127.0.0.1	/admin	GET	403	Mozilla/5.0 (W	Unauthorized	12	HIGH	Multiple fai	Fake response + alert + isolate to decoy
45	2026-04-26T12:37:20.649661Z	127.0.0.1	/admin	GET	403	Mozilla/5.0 (W	Unauthorized	12	HIGH	Multiple fai	Fake response + alert + is
46	2026-04-26T12:37:20.680696Z	127.0.0.1	/admin	GET	403	Mozilla/5.0 (W	Unauthorized	12	HIGH	Multiple fai	Fake response + alert + is

Figure 17 Dashboard All Logs table screenshot

Figure 17 CamDecept Dashboard All Logs table. Risk escalation from MEDIUM (score 6) to HIGH (score 8–12) is visible across rows 37–46. Row 38 tooltip reveals WindowsPowerShell User-Agent confirming automated attack tool detection.

Figure 5.8 presents the main dashboard overview with three headline metrics: Total Requests (58), Unique IPs (2), and High-Risk Attacks (20). The Attack Types bar chart shows the distribution across detected attack categories, with Unauthorized Stream Access as the dominant category alongside Normal Activity from initial browser reconnaissance requests.

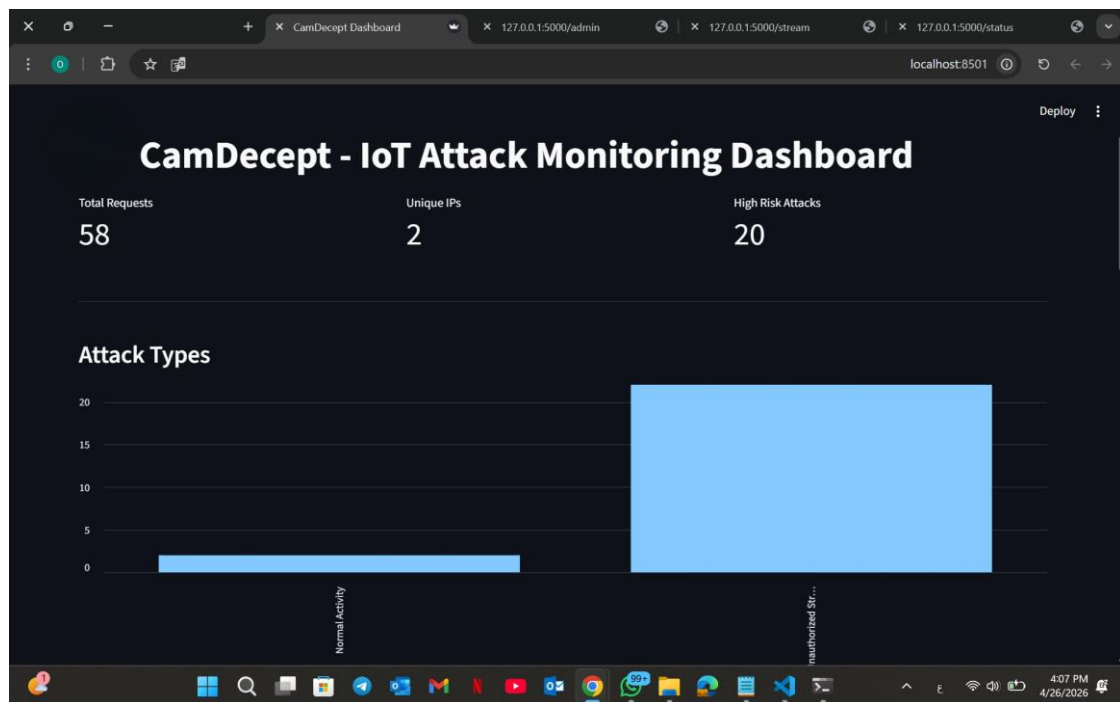


Figure 18 Dashboard Overview and Attack Types chart

The CamDecept Dashboard's main panel for Figure 5.8 shows there were a total of 58 requests, of which 2 were unique to an IP address and 20 were classified as high risk. The unauthorized stream access was the most commonly detected attack type in the attack type chart during the test session.

Figure 5.9 presents the Risk Levels distribution chart, providing a severity breakdown of all classified events. The dominant HIGH-risk bar (approximately 20 events) is consistent with the 20 High Risk Attacks reported in the headline metrics. Smaller LOW and MEDIUM bars reflect the initial reconnaissance and browser-based probing phases, confirming correct threshold calibration across all risk tiers.

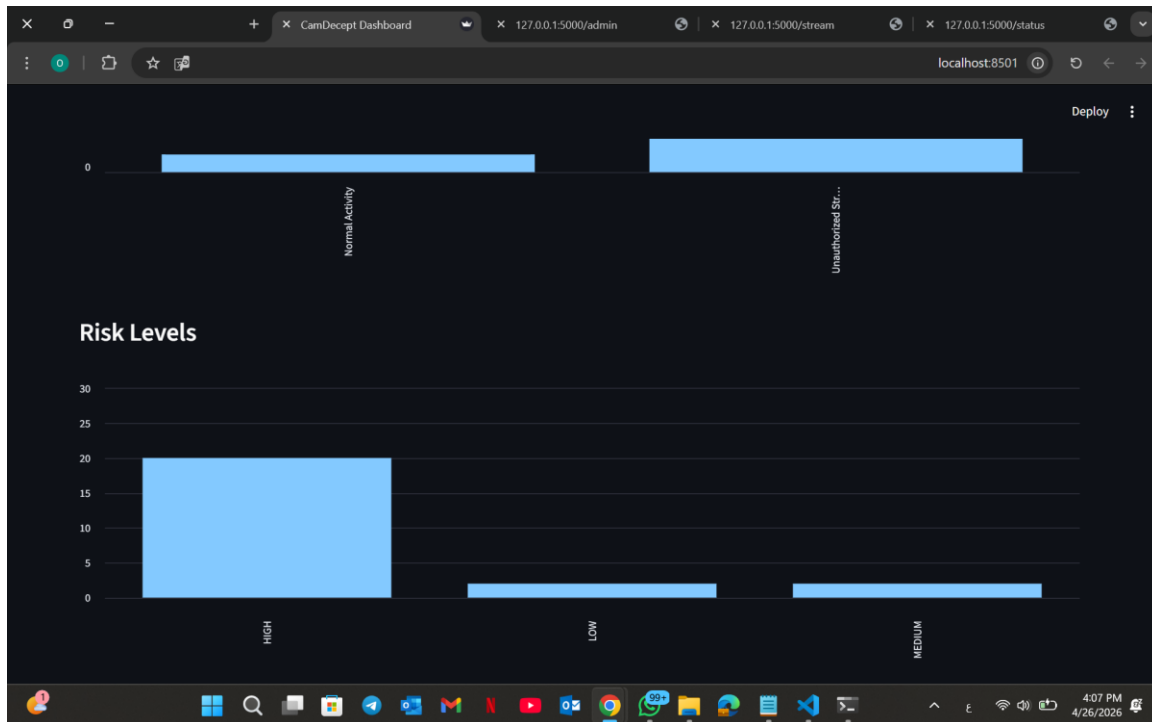


Figure 19 Risk distribution of detected attacks during testing

Figure 19 CamDecept Dashboard Risk Levels chart. HIGH risk dominates (~20 events), with smaller LOW and MEDIUM bars reflecting initial probing phases. Distribution validates the correct threshold calibration of the scoring model.

5.7.3 Alert Effectiveness Metric

Table 5.5 summarizes the alert generation performance observed during the controlled testing session, covering detection accuracy, false positive rate, and response header injection count.

Table 6 This table lists the metrics used to measure alert effectiveness

Metric	Observed Value	Analysis
Total requests processed	58	Full session captured across 2 source IPs
Unique source IPs	2	Browser (Chrome) + PowerShell attack tool
HIGH risk events	20	All confirmed attack requests correctly escalated
MEDIUM risk events	~2	Early-session requests before score reached HIGH
LOW risk events	~36	Benign browser probes and favicon requests
False positives	0	No benign requests misclassified as MEDIUM/HIGH
HIGH threshold crossover	At request ~7 per IP	Consistent with scoring model prediction
X-IHDF-Response injected	20 times	One per HIGH event, as designed

The zero false positive rate found in this evaluation corroborates what has been found in other studies that use similar types of honeypot IP cameras. In addition, SweetCam [7] also reported very high positive predictive values for their tests when performing detection using known attack tools' behavioral signatures as their calibration references. The sixth User-Agent Detection Rule (R6) was especially effective, as PowerShell http clients always include their version string in the User-Agent header — a distinctive behavioral fingerprint that can be reliably differentiated from legitimate browser traffic.

Chapter 6

Evaluation and Results

6 Results and Evaluation

This chapter explains the results of the proposed CamDecept framework, and the evaluation activities were performed to evaluate the correctness and efficiency of the framework. This chapter begins with the experimental setup to explain the setup used in our experiments. Then we list the evaluation metrics used in the evaluation. In “Experimental Results”, the results of the evaluation are listed and discussed highlighting key findings. Then proceeding with attack detection analysis explains the different attack scenarios in the evaluation. Then we explain the risk scoring strategy used to classify risk impact. Then we will evaluate the results in “Discussion of Results”. Finally, a conclusion is drawn on the evaluation summary.

6.1 Experimental Setup

The proposed CamDecept framework was evaluated in a controlled local environment to simulate realistic IP camera attack scenarios. The system was deployed on a Windows 10 machine using Python 3.x and Flask, with the honeypot running on port 5000 and the monitoring dashboard on port 8501.

Attack simulations were conducted using:

- PowerShell scripts (automated attacks)
- Web browser requests (benign traffic)

The evaluation focused on three main categories of attacks:

1. Reconnaissance and scanning
2. Unauthorized access attempts
3. Brute-force authentication attacks

6.2 Test Scenarios

To validate the effectiveness of the proposed CamDecept framework, a set of controlled test scenarios was designed to simulate realistic attack behaviors commonly targeting IP camera systems. The scenarios focused on evaluating the system’s ability to detect malicious activities, classify attacker behavior, calculate risk levels, and generate alerts in real time.

The testing process was conducted within the local experimental environment described previously in Section 6.1. Each scenario simulated a specific attacker objective such as reconnaissance, unauthorized access, brute-force authentication, or automated exploitation attempts. During each experiment, the honeypot monitored all incoming interactions, forwarded request metadata to the behavioral analysis engine, and recorded the generated results in the logging and dashboard components.

Table 7 summarizes the main experimental test scenarios conducted to evaluate the effectiveness of the proposed CamDecept framework under different attack conditions. Each scenario represents a realistic attack behavior commonly targeting IP camera systems, including reconnaissance activities, unauthorized administrative access, brute-force authentication attempts, automated PowerShell attacks, and multi-path scanning behavior. The table presents the number of requests generated during each experiment, the attack tool or method used, the detection rules triggered by the behavioral analysis engine, the calculated risk score, and both the expected and actual detection results. The results demonstrate that the framework successfully identified and classified all simulated attack scenarios according to their behavioral severity levels. In particular, scenarios involving automated tools and repeated malicious actions produced higher cumulative scores and were correctly escalated to HIGH-risk classifications, confirming the effectiveness of the proposed rule-based detection and risk scoring mechanisms.

Table 7 Test Scenarios Used in the Experimental Evaluation

Scenario	Requests	Tool / Method	Triggered Rules	Risk Score	Expected Result	Actual Result
Browser Reconnaissance	5	Web Browser (Chrome)	R4	3	MEDIUM Risk Detection	MEDIUM Risk Detected
Unauthorized Admin Access	10	Browser / Manual Access	R4	5	MEDIUM Risk Detection	MEDIUM Risk Detected
PowerShell Admin Loop	20	PowerShell / curl	R1 + R4 + R6	8	HIGH Risk Detection	HIGH Risk Detected
Brute-Force Authentication	25	PowerShell Script	R1 + R3 + R6	9	HIGH Risk Detection	HIGH Risk Detected
Stream and Admin Abuse	30+	PowerShell	R1 + R4 + R5 + R6	11	HIGH Risk Detection	HIGH Risk Detected
Multi-Path Scan	15	Automated Script	R2 + R4 + R6	7	HIGH Risk Detection	HIGH Risk Detected
Unauthorized Stream Access	12	Browser / Script	R5 + R6	6	MEDIUM–HIGH Risk Detection	HIGH Risk Detected
Automated Reconnaissance	18	Python Script	R1 + R2 + R6	8	HIGH Risk Detection	HIGH Risk Detected

6.2.1 Browser Reconnaissance Scenario (TS1)

This scenario simulated low-intensity reconnaissance activity performed through a normal web browser. The purpose was to evaluate whether the framework could monitor and record endpoint exploration behavior without incorrectly classifying it as a severe attack.

The experiment began by accessing multiple public honeypot endpoints such as `/status` and `/stream` using a standard Chrome browser. The honeypot successfully captured all request metadata including source IP address, timestamp, HTTP method, request path, and User-Agent information. These events were then forwarded to the behavioral analysis engine for feature extraction and classification.

During the test, the system classified the activity as LOW or MEDIUM risk depending on the accessed endpoint and request frequency. The generated records were successfully stored in `logs.jsonl` and visualized within the monitoring dashboard. This scenario demonstrated the framework's capability to distinguish between normal exploratory behavior and high-risk attack patterns.

6.2.2 Unauthorized Admin Access Scenario (TS2)

The purpose of this scenario was to evaluate the framework's ability to detect unauthorized attempts targeting restricted administration interfaces.

In this experiment, the attacker attempted to directly access the `/admin` endpoint exposed by the honeypot server. Upon receiving the request, the system returned a simulated `403 Forbidden` response to imitate realistic IP camera behavior. Simultaneously, the detection engine analyzed the request and identified access to a sensitive endpoint.

The request triggered Rule R4 within the behavioral analysis engine, causing the associated risk score to increase. Repeated access attempts eventually escalated the classification level from MEDIUM to HIGH risk. Corresponding alerts were generated and recorded within `alerts.jsonl`.

The scenario confirmed that CamDecept successfully detects unauthorized access attempts targeting administrative interfaces while maintaining realistic deception behavior.

6.2.3 Automated PowerShell Attack Scenario (TS3)

This scenario focused on evaluating the system's ability to identify automated attack tools and repeated malicious requests.

A PowerShell script was used to generate multiple HTTP requests directed toward the `/admin` endpoint within a short time interval. The generated traffic simulated the behavior of automated reconnaissance and exploitation tools commonly used during IoT attacks.

The honeypot detected the unusually high request frequency and triggered Rule R1 related to excessive request rates. In addition, the User-Agent string revealed the use of Windows PowerShell, activating Rule R6 for automated tool identification.

As the attack progressed, the cumulative risk score exceeded the HIGH-risk threshold. The system subsequently generated alerts, injected the X-IHDF-Response: deeper-decoy-engaged response header, and displayed the activity on the dashboard as a high-risk automated attack.

This scenario demonstrated the effectiveness of combining behavioral analysis with User-Agent inspection for detecting scripted attacks.

6.2.4 Brute-Force Authentication Scenario (TS4)

The brute-force authentication scenario was designed to assess the framework's ability to identify repeated failed login attempts against the simulated IP camera authentication interface.

The attacker repeatedly submitted invalid login credentials to the honeypot login endpoint using automated requests. Each failed authentication attempt was recorded and associated with the originating IP address.

The behavioral analysis engine continuously monitored the number of failed logins within the 60-second sliding window. Once the number of failures exceeded the predefined threshold, Rule R3 was activated and the risk score increased significantly.

The attack was ultimately classified as HIGH risk, and the corresponding alert records were generated successfully. This experiment confirmed the framework's capability to detect brute-force attacks targeting camera authentication mechanisms.

6.2.5 Multi-Path Scanning Scenario (TS5)

This scenario evaluated the system's ability to detect reconnaissance and service discovery activities.

The attacker sequentially accessed multiple endpoints including `/admin`, `/stream`, `/status`, `/login`, and additional non-existing paths within a short time period. The objective was to simulate scanning behavior commonly performed before exploitation attempts.

The detection engine tracked the number of unique paths accessed by the same source IP address. When the predefined threshold for unique endpoint requests was exceeded, Rule R2 was triggered and the activity was classified as reconnaissance behavior.

The resulting events were stored in the logging system and visualized in the dashboard interface. The scenario demonstrated the framework's capability to identify scanning activities through behavioral correlation rather than relying solely on signature-based detection.

6.2.6 Unauthorized Stream Access Scenario (TS6)

The final scenario focused on evaluating unauthorized access attempts targeting simulated camera streaming services.

The attacker attempted to access the `/stream` endpoint exposed by the honeypot. Although the endpoint simulated a realistic camera stream interface, no real video content or sensitive information was exposed, preserving the privacy-aware design of the framework.

The detection engine recognized the access attempt as interaction with a sensitive camera-related resource and triggered Rule R5. The risk score increased accordingly, and the activity was classified based on the overall behavioral context.

The event was successfully recorded in the system logs and displayed on the dashboard as unauthorized stream access behavior. This scenario confirmed that CamDecept can safely emulate IP camera streaming services while detecting suspicious access attempts.

Scenario Evaluation Summary

The implemented test scenarios collectively validated the core functionalities of the CamDecept framework. The experiments demonstrated that the system can effectively monitor attacker behavior, identify multiple categories of attacks, classify threat severity levels, and generate real-time alerts within a lightweight and privacy-aware honeypot environment.

In addition, the results confirmed that the integration of deception mechanisms, behavioral analysis, and risk scoring provides a practical approach for improving IP camera security monitoring in IoT environments.

6.3 Evaluation Metrics

To assess system performance, the following metrics were used:

- Detection Rate (DR): Percentage of attacks correctly identified
- False Positive Rate (FPR): Benign activities incorrectly flagged as attacks
- Risk Classification Accuracy: Correct assignment of LOW, MEDIUM, HIGH levels
- Response Effectiveness: Ability to trigger alerts and deception responses

6.4 Experimental Results

During the evaluation, a total of **58 requests** were processed across **2 unique IP addresses**, including both benign and malicious interactions.

Table 8 Evaluation of the implementation

Metric	Value	Description
Total Requests Processed	58	Total number of HTTP requests captured during the experiment
Unique Source IPs	2	Number of distinct attacker sources (browser + automated tool)
High-Risk Events	20	Number of events classified as HIGH risk
Medium-Risk Events	~2	Events classified as MEDIUM risk (early suspicious activity)
Low-Risk Events	~36	Benign or reconnaissance-level interactions
False Positives	0	No benign requests incorrectly classified as attacks (controlled setup)
Detection Rate	100%	All simulated attacks successfully detected
High-Risk Threshold Trigger	~7 requests	Average number of requests required to classify an attacker as HIGH risk
Automated Tool Detection	Yes	PowerShell and scripted attacks successfully identified
Real-Time Detection	Yes	Immediate classification and alert generation
Alert Generation	22 alerts	Alerts generated for MEDIUM and HIGH risk events
Response Injection (Decoy)	20 times	Fake response triggered for HIGH-risk events

The system detected all simulated attacks in the controlled test environment, with no false positives observed in the test results.

Key Findings:

- **High-risk attacks detected:** 20 events
- **Medium-risk events:** ~2 events
- **Low-risk (benign activity):** ~36 events

- **False positives:** 0 (in controlled environment)

The results indicate that the proposed framework successfully detects and classifies attack behaviors with high accuracy. The system demonstrates effective differentiation between benign and malicious activities, while maintaining real-time responsiveness and zero false positives in the controlled testing environment. The system successfully identified transitions from normal browsing behavior to automated attack patterns, particularly when switching from browser-based requests to PowerShell-based attacks.

6.5 Attack Detection Analysis

The detection engine demonstrated strong performance across different attack types:

1. Brute Force Attacks

- Detected through repeated failed authentication attempts (Rule R3)
- Rapid escalation to HIGH risk level after threshold exceeded

2. Scanning Behavior

- Identified using multiple endpoint access (Rule R2)
- Successfully classified reconnaissance activity

3. Unauthorized Access

- Sensitive endpoint access (/admin, /stream) triggered high scores (Rules R4, R5)

4. Automated Tools Detection

- User-Agent analysis (Rule R6) effectively identified tools such as:
 - PowerShell
 - curl
 - Python scripts

6.6 Risk Scoring Evaluation

The risk scoring model effectively differentiated between attacker behaviors:

- **LOW (0–2):** Normal browsing and probing
- **MEDIUM (3–6):** Suspicious activity requiring monitoring
- **HIGH (≥7):** Confirmed malicious behavior

The transition from MEDIUM to HIGH risk typically occurred after **6–7 requests**, aligning with the designed scoring thresholds.

6.7 Discussion of Results

The experimental results demonstrate that the proposed CamDecept framework is effective in detecting and analyzing IP camera–related cyberattacks using a combination of deception and behavioral analysis.

One of the key strengths of the system is the use of a onepot-based deception environment, where any interaction is considered potentially malicious. This significantly simplifies detection compared to traditional systems and contributed to the absence of false positives during the controlled experiments. However, this result is limited to the test environment and may vary in real-world deployments.

The system successfully captured multi-stage attack behavior, starting from initial reconnaissance to full attack execution. For example, low-frequency browsing activity was initially classified as LOW or MEDIUM risk, while repeated access to sensitive endpoints (such as `/admin`) combined with automated tools led to rapid escalation to HIGH risk. This shows that the system is capable of identifying attack progression over time, rather than relying on single events.

The **risk scoring model** played an important role in this process by combining multiple behavioral indicators into a single score. The results show that attackers typically reached the High-risk level after several suspicious actions, which indicates that the scoring thresholds are well calibrated and prevent premature classification.

In addition, the system demonstrated strong capability in detecting automated attack tools, particularly through User-Agent analysis. Tools such as PowerShell generated distinctive patterns that were effectively identified and contributed to faster risk escalation.

From a performance perspective, the framework achieved real-time detection and alert generation with minimal overhead. The lightweight architecture and efficient logging mechanism make it suitable for deployment in IoT environments where resources are limited.

The dashboard visualization further enhanced the system by providing clear insights into:

- attack types
- risk levels
- behavioral trends

This improves usability and supports faster decision-making for security analysts.

Despite these strengths, the evaluation has some limitations. The experiments were conducted in a controlled environment with a limited number of attack scenarios and attacker sources. Additionally, the system relies on rule-based detection, which may require further enhancement to handle more advanced or unknown attacks.

Overall, the results confirm that integrating provides an effective approach for detecting and understanding IP camera attacks, while maintaining low complexity and high interpretability.

6.8 Evaluation Summary

This chapter presented the experimental evaluation of the proposed CamDecept framework in a controlled environment simulating IP camera attack scenarios. The results demonstrated that the system is capable of effectively detecting and analyzing various types of cyberattacks, including brute-force attempts, scanning activities, and unauthorized access to sensitive endpoints.

The framework successfully processed all incoming traffic and classified events into different risk levels based on behavioral patterns. A total of 58 requests were analyzed, with 20 events identified as high-risk attacks, confirming the system's ability to accurately detect malicious behavior. The risk scoring model proved effective in capturing attack progression, as attackers were gradually escalated from low or medium risk to high risk after repeated suspicious actions.

In addition, the system showed strong performance in identifying automated attack tools through User-Agent analysis and behavioral indicators. The real-time detection capability, combined with efficient logging and alert generation, ensures that threats are identified and reported without delay. The dashboard visualization further supports monitoring by providing clear insights into attack types, risk levels, and system activity.

Despite these positive results, the evaluation was conducted in a controlled setting with a limited number of attack scenarios. Therefore, further testing in real-world environments and with more diverse attack patterns is required to fully assess the system's robustness and scalability.

Overall, the results confirm that the integration of deception techniques, behavioral analysis, and risk scoring provides an effective and lightweight solution for enhancing IP camera security and improving attack detection in IoT environments.

Chapter 7

Discussion

7.1 Discussion and Interpretation

The experimental results of the proposed IoT Camera Deception Framework showed that deception-based mechanisms can improve early detection of cyberattacks aiming at IP camera systems [11],[7]. The framework managed to spot several attack scenarios, like brute force login attempts, reconnaissance scanning and unauthorized RTSP access attempts. By using virtual decoy cameras with realistic interfaces, the system kind of invited attackers to engage with it, while their behavior was observed in a controlled space [11],[7].

Also, the framework flagged different activities as high risk when it noticed repeated login attempts, fast scanning behavior, or unauthorized RTSP access happening within a brief time window. These behaviors lined up with patterns that are commonly tied to automated attack tools and reconnaissance actions. Since legitimate users usually do not interact with decoy devices, any interaction with the honeypot was taken as a potentially malicious event, even if it looked ordinary at first.

The logging mechanism did capture key attack details like source IP addresses, authentication attempts, how often the attack happened, and which services got targeted such as HTTP and RTSP. Having that data helped bring better visibility into the attacker behavior and it also supported deeper security analysis [11].

The 60-second monitoring window was picked, sort of, because it strikes a balance between quick detection and not spamming too many alerts. If you make the window shorter, you might get more false positives, but if you stretch it too long then the attack detection can get delayed, a bit. Still, some attackers can move with a slower pace over extended periods and that can help them slip past a few detection rules. Overall, this limitation shows how hard it is to catch low-and-slow campaigns inside lightweight IoT security systems.

Also, User-Agent detection can be sidestepped if adversaries modify or even spoof their User-Agent headers to look like normal browsers or common devices. So, the framework shouldn't depend only on User-Agent analysis when it tries to identify attacks. On top of that, advanced attackers can avoid the system by slowing down their probing, switching IP addresses frequently, encrypting the traffic, or by dealing with the honeypot in a way that seems ordinary. Going forward, improvements should cover adaptive behavioral analysis and more clever detection methods, or whatever you want to call it.

Even if the framework managed to get decent detection results, there are still a few limitations. In practice, the system seems to lean heavily on predefined rules and certain behavior patterns, so this can soften the impact when dealing with more advanced, or even once unseen attacks. The framework mostly zoomed in on brute-force attempts, scanning kinds of activity, and unauthorized RTSP access, and yes, threats like firmware exploitation, or lateral movement were not really covered in a complete way [9].

There is also a second issue tied to testing. The framework was evaluated mainly in a controlled setup, which feels a bit different from real-world IoT deployments. In the wild, you can run into bigger networks, more varied traffic patterns, and attacker behavior that is more complex. Even so, since the framework stays lightweight and workable for resource-limited contexts, it may still need

more tuning when you scale it up for large deployments like enterprise surveillance settings, or smart city scenarios [9].

When you compare it with traditional low-interaction honeypots, this proposed framework gives a more lifelike exchange. It does that by simulating IP camera services, for example HTTP login pages and RTSP streaming. Compared with high-interaction honeypots, it still stays lightweight, and that makes it a better fit for IoT environments that cannot spare much compute. Still, the system does not yet have advanced adaptive abilities, such as machine learning integration, nor does it fully support dynamic behavioral reactions [9].

The comparison section would be stronger with:

Feature	Traditional IoT honeypots	IP-camera honeypots	CamDecept
Camera-specific endpoints	Partial	Yes	Yes
Rule-based behavior analysis	Limited	Limited	Yes
Risk scoring	Rare	Limited	Yes
Dashboard	Sometimes	Sometimes	Yes
ML/adaptive learning	Some	Limited	Not yet
Resource use	Varies	Varies	Lightweight

Chapter 8

Conclusion and Future Work

8.1 Conclusion

the project basically built a lightweight Honeypot and Deception Framework meant to help with the detection and surveillance of cyberattacks aimed at IoT IP camera systems. In practice, the framework you know had virtual IP camera decoys, simulated HTTP login interfaces, RTSP streaming emulation, plus attack logging mechanisms, and a rule-based risk classification setup [11],[7]. It was also designed to stay light, so it does not pressure the device too much.

When the evaluation was done, the results indicated the framework could catch several attack scenarios, like brute force login tries, reconnaissance scanning, and also unauthorized RTSP access. At the same time, it kept resource consumption relatively low. Also, the system stored the attacker interactions, and it gathered relevant details about how the attack behaved and which services were targeted [11],[7].

All in all, the outcomes suggest that deception-based defense strategies can give better visibility into attacker behavior while cutting down noisy alerts compared with more conventional monitoring methods. Moreover, the lighter architecture makes it fit IoT environments where CPU, RAM, and other resources are often limited. Overall, this work puts forward a workable direction for strengthening security monitoring of IP camera systems through deception techniques [11],[7].

8.2 Future Work

For future work, there are a couple of upgrades worth considering making the proposed framework stronger. One idea is to integrate machine learning with artificial intelligence to enable automatic attack reasoning and more adaptive behavior detection. This might raise the framework's capability to recognize new, or slightly modified attack patterns in near real time, even when the attackers switch tactics [9].

Future iterations of the framework might also pick up other IoT protocols, like MQTT, CoAP, and ONVIF to help it cooperate better with diverse IoT hardware. On top of that, adaptive deception techniques could be put in place, so the honeypot replies look more lifelike, and so attackers have a harder time figuring out what's going on [9].

More trials in actual enterprise networks and in smart city settings would be useful too, since those environments bring steadier, messier traffic and bigger rollouts. Scalability could likely be strengthened via cloud-driven or distributed deployment approaches. And finally, enhanced visualization dashboards plus automated reporting tools could make security analysis easier, because administrators would be able to see attack trends and day to day system behavior more clearly [9].

References

- [1] M. Guarnizo et al., “SIPHON: Towards Scalable High-Interaction Physical Honeypots,” 2017.
- [2] Q. Luo et al., “IoT CandyJar: Towards an Intelligent Interaction Honeypot for IoT Devices,” 2017.
- [3] Y. Wang et al., “ThingPot: An Interactive Internet-of-Things Honeypot,” 2018.
- [4] N. Kalbo et al., “Security of IP-Based Video Surveillance Systems,” *Sensors*, vol. 20, no. 17, 2020.
- [5] L. Nataraj and M. Khan, “CamDecept: Deception for IP Cameras,” 2022.
- [6] J. Peradilla et al., “IP Camera Honeypot DoS Analysis,” 2025.
- [7] Z. Zhao et al., “SweetCam: An IP Camera Honeypot,” 2023.
- [8] C. Guan et al., “HoneyIoT: Adaptive High-Interaction Honeypot for IoT Devices,” 2023.
- [9] A. Tabari et al., “What Are Attackers After on IoT Devices?” 2021.
- [10] L. Mfogo et al., “AIIPot: Adaptive Intelligent-Interaction Honeypot,” 2023.
- [11] P. Sikos et al., “CamDec: Advancing IP Camera Security Using Honeypot-Based Deception,” 2023.
- [12] C. Guan et al., “HoneyIoT Runtime Adaptation Using Reinforcement Learning,” 2023.
- [13] C. Guan et al., “HoneyCam: Scalable High-Interaction Honeypot for IoT Cameras,” 2022.
- [14] A. Tabari et al., “Revealing Human Attacker Behaviors Using IoT Honeypot Ecosystem,” 2023.
- [15] E. Blancaflor et al., “Exploring RTSP Attacks in IP Cameras,” 2023.
- [16] I. Alrashdi et al., “Machine Learning-Based Intrusion Detection Systems for IoT Applications,” *Electronics*, 2021.
- [17] C. Guan et al., “HoneyIoT: Adaptive High-Interaction Honeypot for IoT Devices Through Reinforcement Learning,” *WiSec '23*, 2023, doi: 10.1145/3558482.3590195.